Streamlining Transformer-based Detection and RL-based Response for Automated Threat Mitigation

Student: Adrian Santoso

Advisor: Prof. Ying-Dar Lin

High Speed Network Lab

National Yang Ming Chiao Tung University, Taiwan

Outline

- Motivation
- Background
  - Security Operations Center
  - MITRE ATT&CK Framework
  - Cyber Threat Intelligence
  - CTI-to-Lifecycle Extraction with MITREtrieval
  - Transformer-Based Technique Detection
  - Reinforcement Learning for Automated Response
  - AI Agent and Automation Architectures
- Related works
- Issues
- Notations
- Problem Statement
- Solution
  - Transformer-Based Technique Detection
  - RL State, Action Space, and Reward Function
  - Transformer+RL vs. LLM Detection–Response
- Implementation
- Evaluation
- Schedule
- References

Motivation

1. Excessive Alert Volume Leads to Operational Inefficiency
   - Most alerts are low-impact
   - Analysts spend too much time reviewing them → delays detection of real threats

2. Incident Response Still Relies on Manual Work
   - In current tools, actions must be selected, configured and triggered manually
   - Cause slow response time (high MTTR) and gives attackers more time

3. Need AI-Driven Solution
   - AI agent can detect threats and trigger response in real time
   - Speeds up action and stopping the attack

Background – Security Operations Center (1/7)

- Security Operations Center (SOC) monitors, detects, and responds to cyber threats 24/7
- Main data sources: IDS, Firewall, EDR, NDR
- **Core platforms:**
  - SIEM: collects and correlates security events
  - SOAR: supports orchestration and response automation
- **Main roles of SOC analysts:**
  - Monitor systems and networks continuously
  - Prioritize incidents and response actions
- **Key challenges:**
  - Large alert volume
  - Slow manual investigation and response

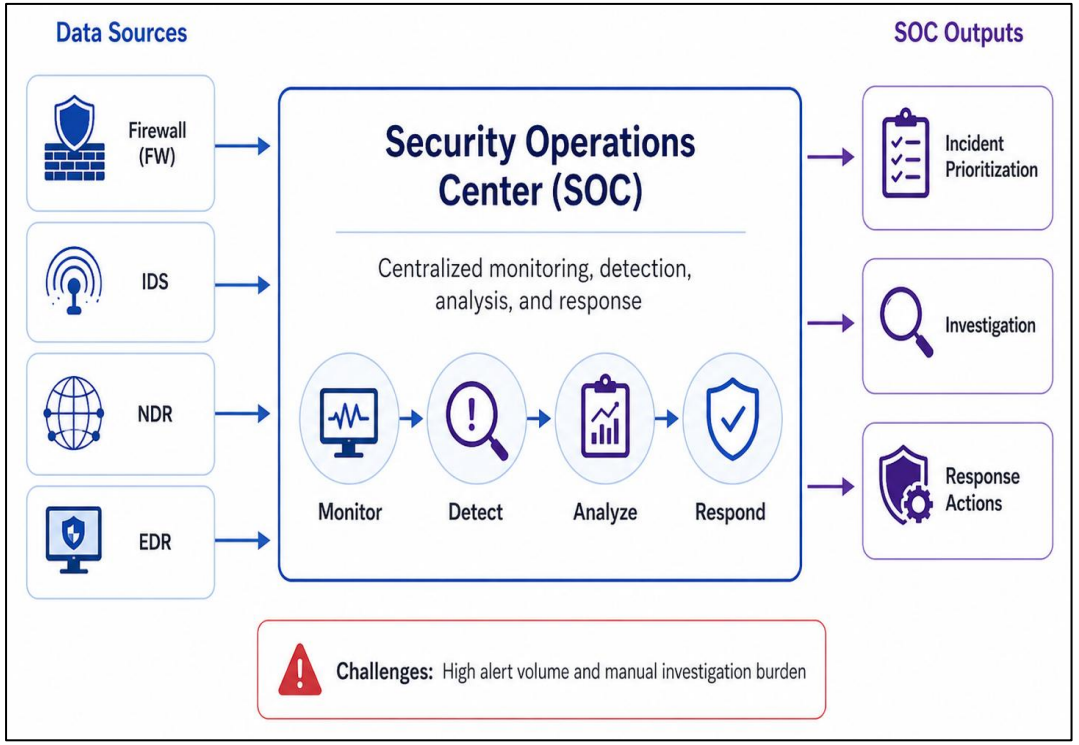


Figure. Security Operations Center workflow for centralized cyber defense

Background – MITRE ATT&CK Framework (2/7)

- MITRE ATT&CK is a globally adopted knowledge base of real-world attacker behaviors
- **It organizes adversarial behavior into:**
  - Tactic: attacker goal
  - Technique: how the attacker performs the action
  - Procedure: real-world example of technique usage
- A sequence of techniques can be viewed as an attack lifecycle
- **Why it is important:**
  - Provides a common language for threat analysis
  - Helps map alerts to attacker behavior

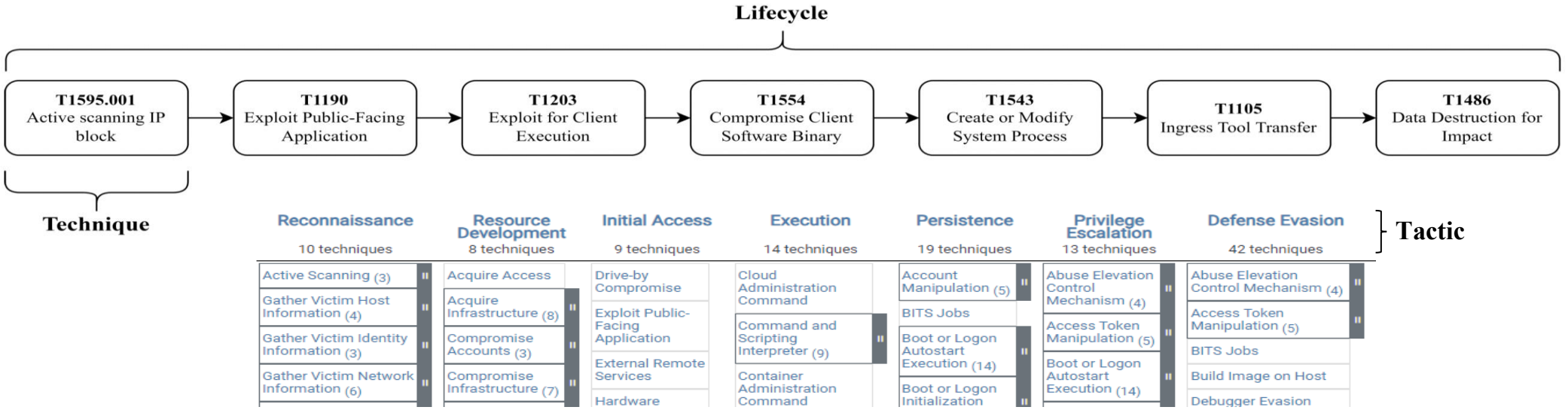


Figure. MITRE ATT&CK Framework

Background – Cyber Threat Intelligence (3/7)

- Cyber Threat Intelligence (CTI) gathers and analyzes threat-related information to improve defense
- **CTI provides actionable knowledge for:**
  - Threat detection
  - Response planning
- **Common CTI information includes:**
  - Indicators of Compromise (IoCs): IPs, domains, URLs, hashes
  - Tactics, Techniques, and Procedures (TTPs): attacker methods and behavior
- **Why CTI is important:**
  - Connects raw security events to attacker intent
  - Provides external knowledge beyond local logs

Background – CTI-to-Lifecycle Extraction with MITREtrieval (4/7)

- **Goal:** extract MITRE ATT&CK techniques from unstructured CTI reports
- **Key idea:**
  - Uses deep learning to identify attacker techniques from CTI Report
- **Why it is important:**
  - Contain rich attacker behavior
  - Structured lifecycles are also useful for attack simulation
- **Role in this work:**
  - Extracted lifecycles are used to build attack scenarios
  - Used to support detection model fine-tuning

Unstructured CTI Report

Text
GravityRAT - The Two-Year Evolution Of An APT Targeting India This blog post is authored by Warren Mercer and Paul Rascagneres. Since the publication of the blog post, one of the anti-VM capability was commented a lot on Twitter: the detection of Virtual Machines by checking the temperature of the system. We decided to add more details and clarifications concerning this feature. GravityRAT uses a WMI request in order to get the current temperature of the hardware. Here is the output of the query on a physical machine (a Surface Book): The query returns the temperature of 7 thermal zones. Here is the output on a Virtual Machine executed by Hyper-V on the same hardware: The feature is not supported. The malware author used this behavior in order to identify VM (such as Sandboxes). From our tests and the feedback from several researchers, this monitoring is not supported on Hyper-V, VMWare Fusion, VirtualBox, KVM and XEN. It's important to notice that several recent physical systems do not support it (a researcher reported some Lenovo and Dell hosts did not support this). It means that GravityRAT will consider this physical machine as VMs. Importantly to note this check is not foolproof as we have identified physical hosts which do not report back the temperature, however, it should also be considered a check that is identifying a lot of virtual environments. This is particularly important due to the amount of sandboxing & malware detonation being carried out within virtual environments by researchers. Summary Today,

Converting Report to Technique

Text	T1047	T1113	T1037	T1033
GravityRAT -	1	0	0	1

Figure. Extracted MITRE ATT&CK Technique Lifecycle

Background – Transformer-Based Technique Detection (5/7)

- **Transformer Models:**
  - Neural network for sequential text
  - Learns contextual relationships
- **Why Transformer-Based Models:**
  - Alerts and CTI are mostly textual
  - Strong at context understanding
  - Has lower latency than LLMs → better for real-time SOC use
- **Basic Detection Idea:**
  - Collect alerts from Firewall, IDS, NDR, and EDR
  - Detect the current MITRE ATT&CK technique ID

Background – Reinforcement Learning for Automated Response (6/7)

- **Reinforcement Learning (RL):**
  - Learns actions through environment interaction and rewards
  - Used here to select response actions after technique detection
- **Why RL fits this problem:**
  - Response is a sequential decision problem
  - One action can change future system states
- **Response objective:**
  - Restore the system to a normal state
  - Block or reduce future attacker techniques
  - Avoid unnecessary disruption
- **Why it is important:**
  - RL learns adaptive policies better than fixed rules

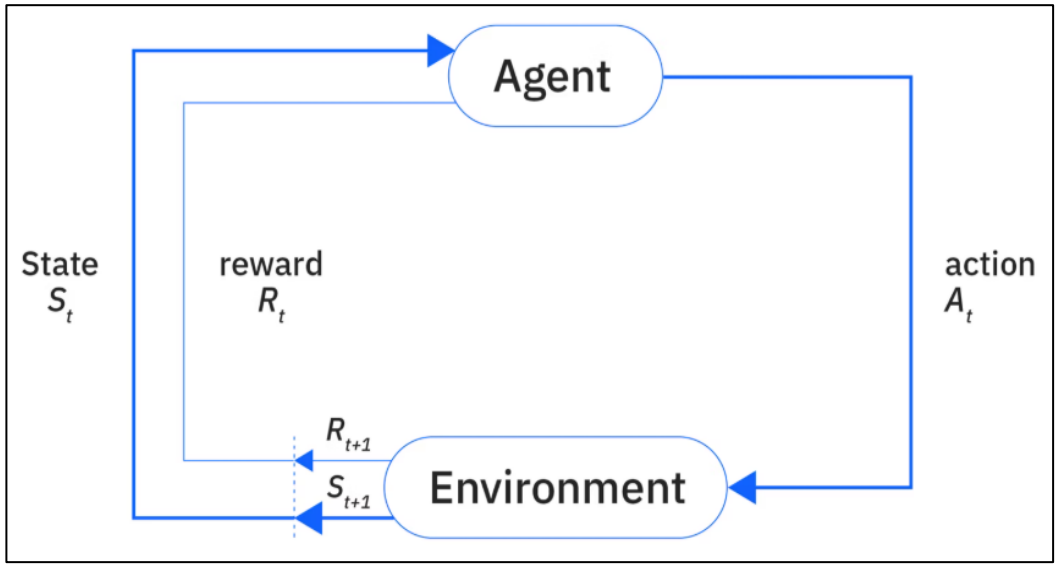


Figure. Reinforcement learning feedback loop between an agent and environment.

Background – AI Agent and Automation Architectures (7/7)

- **AI agent:**
  - Can use tools and external services
  - Can execute actions autonomously in an environment
- **Typical agent capabilities include:**
  - Calling APIs
  - Executing scripts
- **Why it is important here:**
  - The proposed framework is not only about detection
  - It aims to connect detection and response into an automated workflow

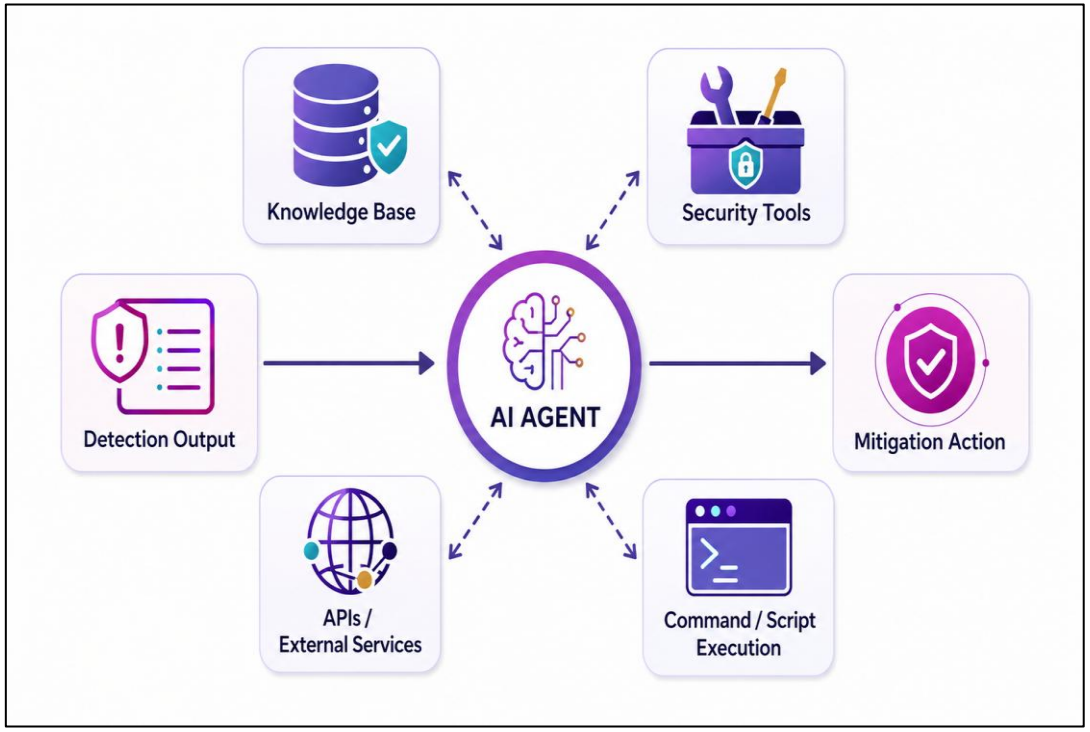


Figure. AI agent-based cybersecurity automation architecture

Related Works on Detection & Response

<table>
<tr><th rowspan="2">Paper</th><th rowspan="2">Study Focus</th><th colspan="2">Detection</th><th colspan="3">Response</th><th rowspan="2">Model</th><th rowspan="2">Dataset</th></tr>
<tr><th>Attack Type Classification</th><th>Alert Correlation</th><th>Automatic</th><th>Risk-aware Response</th><th>Stage-aware Response</th></tr>
<tr><td>[4]</td><td rowspan="3">Detection</td><td rowspan="2">V</td><td>V</td><td rowspan="3">-</td><td rowspan="5">-</td><td rowspan="9">-</td><td>LLM</td><td>IDS, Firewall, EDR, NDR</td></tr>
<tr><td>[5]</td><td rowspan="7">-</td><td>SL</td><td>IDS</td></tr>
<tr><td>[18]</td><td rowspan="6">-</td><td>DL</td><td>IDS</td></tr>
<tr><td>[7]</td><td rowspan="5">Response</td><td rowspan="5">V</td><td rowspan="3">RL</td><td>-</td></tr>
<tr><td>[6]</td><td>Network</td></tr>
<tr><td>[19]</td><td rowspan="3">V</td><td>Rule-based</td><td>IDS</td></tr>
<tr><td>[20]</td><td>RL</td><td>Endpoint</td></tr>
<tr><td>[8]</td><td rowspan="2">Detection + Response</td><td>V</td><td>V</td><td>-</td><td>LLM</td><td>IDS, EDR</td></tr>
<tr><td>Ours</td><td>V</td><td>V</td><td>V</td><td>V</td><td>V</td><td>Transformer + RL</td><td>IDS, Firewall, EDR, NDR</td></tr>
</table>

Table. Survey on Threat Detection and Response

SL: Supervised Learning; UL: Unsupervised learning; DL: Deep Learning; RL: Reinforcement Learning; LLM: Large Language Models

Issues

- Technique Detection: Detect Attacker Movement
  - Compare LLMs and transformers in technique detection using F1-score, accuracy, recall, and inference time
- Response Decision: Action Selection
  - Measure the success rate of RL actions in restoring the system to a normal state across episodes
  - Measure where the attack lifecycle stops across different stage designs
  - Analyze whether the RL agent selects stage-appropriate response actions
- Response Model Comparison: LLM vs RL
  - Compare RL and LLM response in recovery consistency
  - Compare their lifecycle stopping position and stage-aware action behavior
- Architecture Comparison: Transformer+RL vs. LLM Detection–Response
  - Compare both architectures using detection latency, response latency, and end-to-end latency

Notation (1/2)

Category	Symbol	Description
CTI	$L = \{l_i \mid i = 0, \ldots, N_{L-1}\}$	Finite set of lifecycles
	N_L	Number of available lifecycles
	$l_i = \{T_{i,0}, \ldots, T_{i,N_i-1}\}$	Lifecycle l_i as an ordered sequence of techniques
Alert & Detection Module	R = {1: FW, 2: IDS, 3: NDR, 4: EDR}	Index set of four sensor types: Firewall (FW), IDS, NDR, and EDR
	$E_{t,R}$	Single security event generated by sensor R at timestamp t
	W_m	The m-th time window of the alerts
	X_m	Multi-source alert set in time window W_m
	f_θ	Detection model parameterized by θ
	$T_m^{(l)}$	Ground-truth technique at step m in lifecycle l
	$\hat{T_m^{(l)}}$	Technique detected at step m for lifecycle l

Table. Notation Table

Notation (2/2)

Category	Symbol	Description
Response Module	$Z = \{1: \text{NORMAL}, 2: \text{COMPROMISED}\}$	Set of possible system status values
	$G = \{1: \text{EARLY}, 2: \text{MID}, 3: \text{LATE}\}$	Set of possible attack stages
	N_A	Number of predefined actions
	$\text{A} \in \{A_0, \cdots, A_{N_{A-1}}\}$	Set of predefined actions
	$St_m^{(l)}$	System state at step m for detected technique in lifecycle l
	$G_m^{(l)}$	Current stage at step m in lifecycle l
	$F_m^{(l)}$	System artifact set at step m in lifecycle l
	$Z_m^{(l)}$	System status at step m in lifecycle l
	$R_m^{(l)}$	Reward received after response action at step m in lifecycle l
	$a_m^{(l)} \in \text{A}$	Action selected by the RL model at step m in lifecycle l
	π_ϕ	RL response policy parameterized by ϕ
	$Prompt^{(l)}$	LLM response prompt for lifecycle l

Table. Notation Table

Problem Overview

1. Technique Detection: Detect Attacker Movement
   - Task: Detect the attacker's technique from multi-source security alerts
   - Goal: Achieve accurate and low-latency detection
2. Response Decision: Action Selection
   - Task: Select appropriate response actions based on the detected technique and system state
   - Goal: Restore the system, stop the attack early, and respond according to attack stage

Problem Statement – Technique Detection: Detect Attacker Movement

- Input:
  - Multi-source alert set X_m in time window
  - List CTI lifecycle: $L\ \{l_0, l_1, \cdots, l_{N_{L-1}}\}$
- Output:
  - Detected technique $\hat{T}_m^{(l)}$
- Objective:
  - Maximize F1-score, accuracy and recall between detected technique $\hat{T}_m^{(l)}$ and ground-truth $T_m^{(l)}$
- Constraints:
  - Total available lifecycle: N_l
  - Available sensor R: Four sensor types (FW, IDS, NDR, EDR)

Problem Statement – Response Decision: Action Selection

- Input:
  - Detected technique $\hat{T}_m^{(l)}$
  - Current attack stage $G_m^{(l)}$
  - System Artifact set $F_m^{(l)}$
  - System status $Z_m^{(l)}$
  - Predefined action set A
- Output:
  - Selected action: $a_m^{(l)}$
- Objective:
  - Maximize recovery success and early lifecycle stopping through stage-aware response action selection
- Constraints:
  - Chosen actions $a^l \in \{A_0, A_1, \cdots, A_{N_{A-1}}\}$

Solution Overview

- **Attacker Interaction**: External attacker interacts with internal systems
- **Security Tools**: Generate alerts E from security tools
- **Detection Agent**: Performs alert correlation, detects the current technique $\hat{T}_m^{(l)}$
- **Response Agent**: Recommends and executes selected actions $a_m^{(l)}$ (ex: block IP, isolate host, etc)

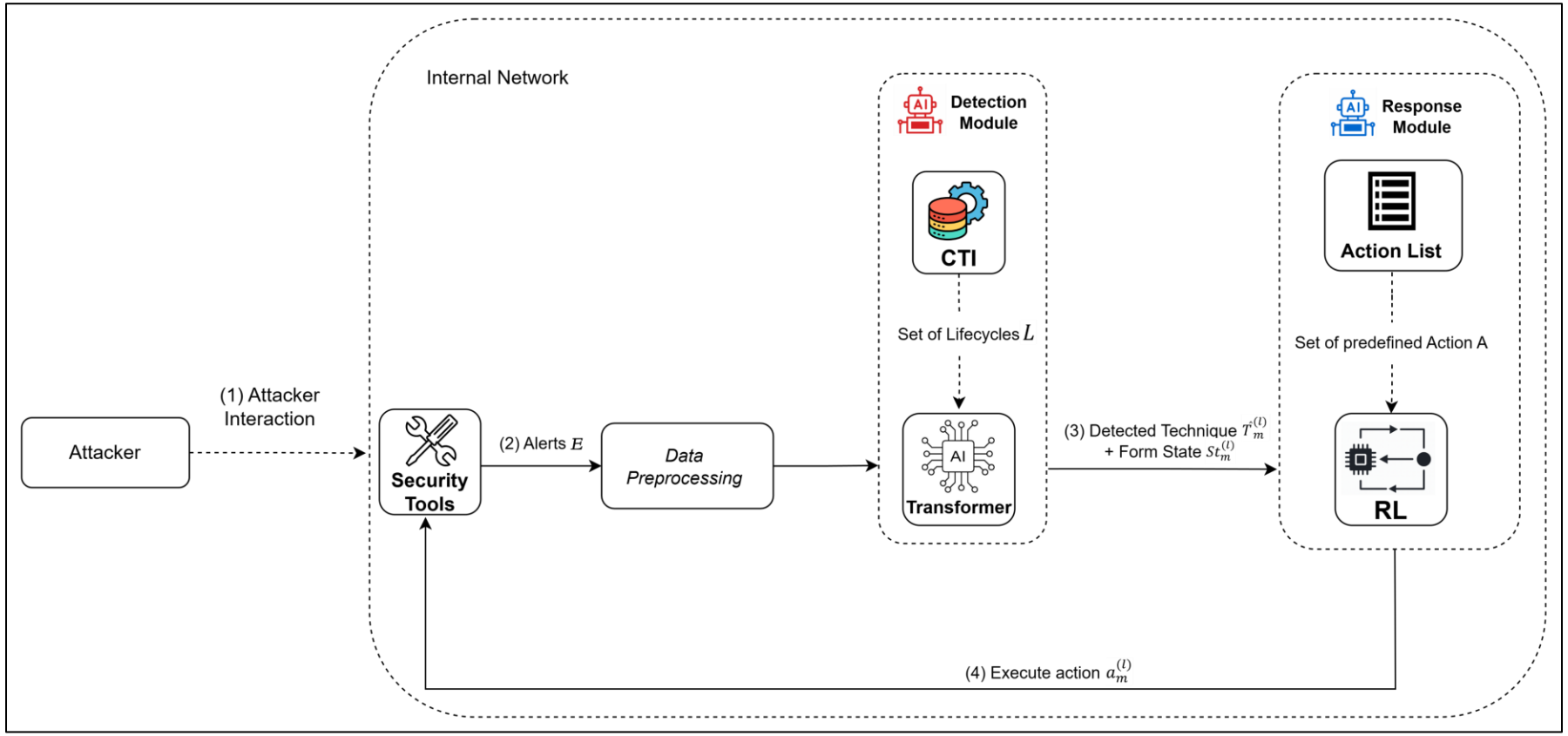


Figure. Solution Overview

Solution – Transformer-Based Technique Detection

1. Attack execution is performed continuously, with an interval between technique
2. Collect multi-source alerts E (FW, IDS, NDR, EDR).
3. Normalize and order events by timestamp
4. Feature selection: Extract key features from alerts
5. Split alerts into time windows X_m
6. Model Input: Each time-windowed alert group X_m
7. Detection Model detect the current technique $\hat{T}_m^{(l)}$

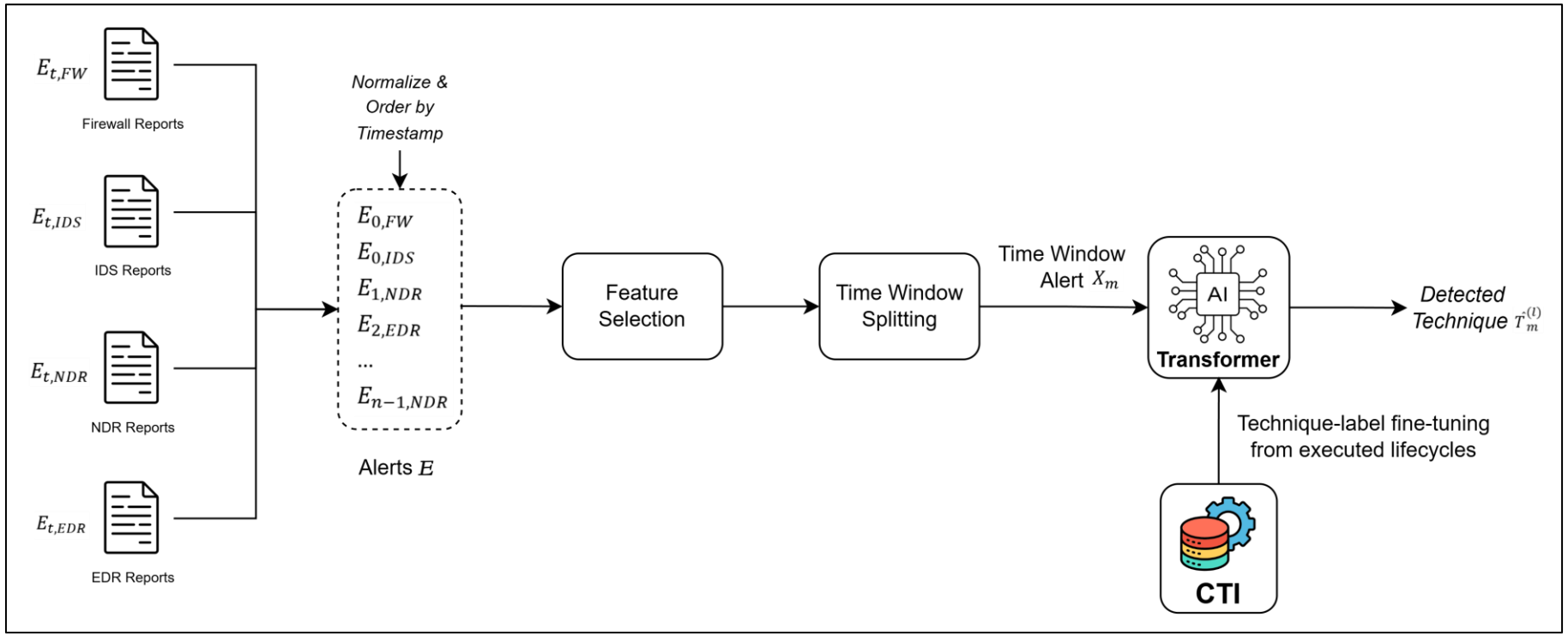


Figure. Transformer-Based Technique Detection

Solution – RL State

- State Components $St_m^{(l)}$:
  - Current Stage $G_m^{(l)}$: Attack stage based on technique progression
  - System Artifact $F_m^{(l)}$: Compromise-related artifacts on the host
  - System Status $Z_m^{(l)}$: Whether the host is NORMAL or COMPROMISED

- Why These:
  - Current Stage : Capture attack progression for context-aware decisions
  - System Artifact: Identifies remaining compromised artifacts, enabling targeted remediation actions
  - System Status: Provides the final recovery outcome for RL reward and evaluation

Solution – Response Action Space

Impact Level	Name	Description
Low	Force sign out	Logs out a user from all active sessions to stop unauthorized activities.
	Process cleanup	Terminates malicious process artifacts.
	File cleanup	Removes malicious files and staged collection artifacts.
	Registry cleanup	Reverts malicious registry key changes.
	Service / Task cleanup	Removes malicious service and scheduled task artifacts.
	Reset TCP connection	Drops or resets a suspicious network connection.
Medium	Force password reset	Forces a user to reset their password to mitigate compromised credentials.
	Disable user account	Disables a user account to stop unauthorized access.
	Add IP to block list	Adds a specific IP address to the firewall block list to stop suspicious network communication.
	Restrict command execution	Blocks or limits command interpreter and scripting tools such as cmd.exe, PowerShell and WMIC.
	Restrict write operations	Blocks or limits file writes, registry edits, service creation, and scheduled task creation.
High	Shutdown workstation	Shuts down an asset to contain and mitigate the attack.
	Isolate endpoint	Disconnects an asset from the network to contain and mitigate the attack.

Table. RL Action Space

Solution – RL Reward Function

1. Outcome Reward Table

System Status	Reward	Meaning
NORMAL	+2.0	System restored successfully
COMPROMISED	-1.5	System remains compromised

2.1 Stage-Aware Action Reward & Penalty

Stage	Low Impact	Medium Impact	High Impact
Early	+0.5	0.0	-2.0
Mid	0.0	+0.5	-1.0
Late	-0.5	+0.5	+1.0

2.2 Non-Stage Action Reward & Penalty

Low Impact	Medium Impact	High Impact
+0.5	0.0	-1.5

3. Technique Reduction Reward

Reward Rule	Meaning
• blocked_count_technique × 1 • max = 6	Encourages technique blocking while limiting over-reward for high-impact actions

Solution – Transformer+RL vs. LLM Detection–Response

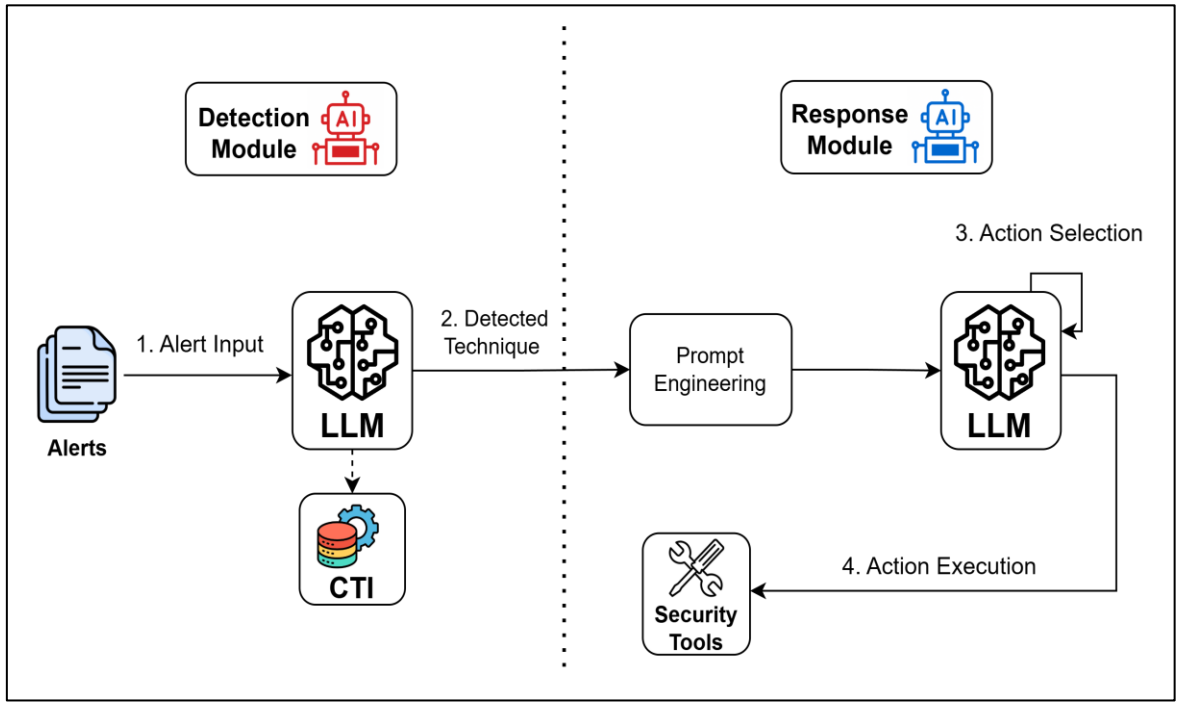


Figure. LLM-based detection & response

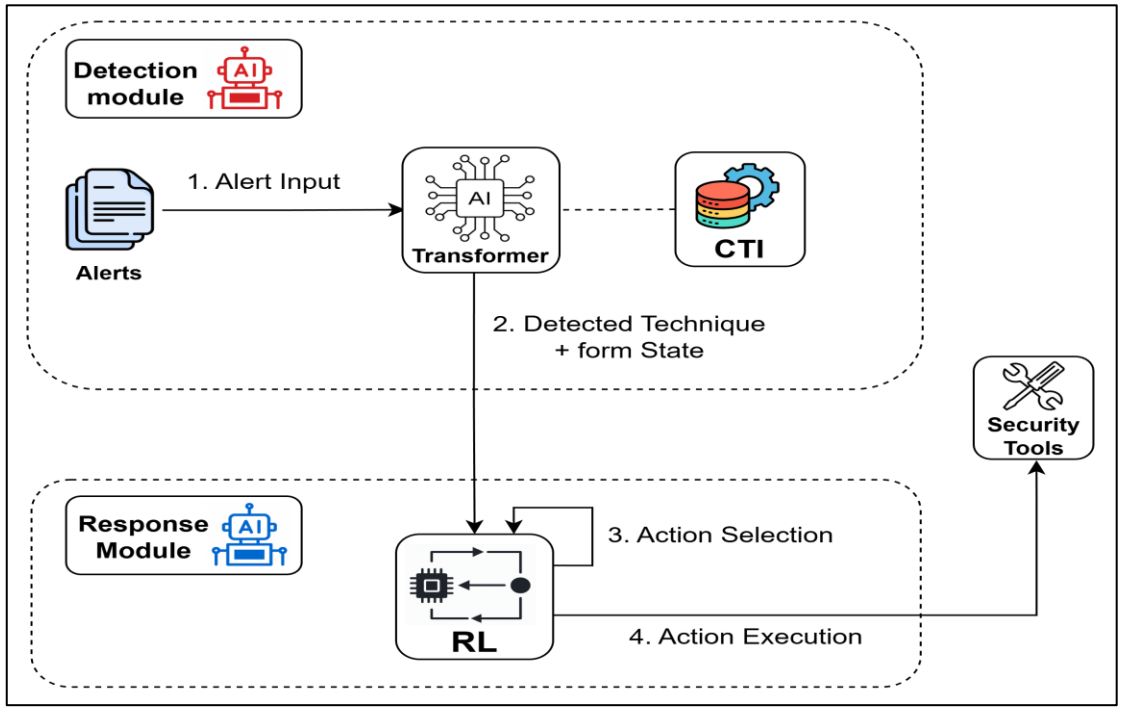


Figure. Transformer-based detection and RL-based response

LLM-based detection & response

1. LLM handles both detection and response
2. The CTI-fine-tuned LLM detects the current technique
3. System state is formed before prompt engineering
4. The LLM selects and executes the response action

Transformer-based detection and RL-based response

1. Two models are used: Transformer and RL
2. The CTI-fine-tuned Transformer detects the current technique
3. The detected technique and system state are the RL input
4. The RL agent selects and executes the response action

Implementation – Open-Source Tools and Libraries

Category	Name	Description & Functionality
Virtualization Tools	VMware Workstation	Runs isolated virtual environments for testing.
Security Tools	OPNsense (Firewall)	Filters traffic, applies network rules.
	Suricata (IDS)	Detects and blocks malicious network traffic.
	Wazuh +Suricata (NDR)	Analyzes network flows for threats.
	Wazuh + Sysmon (EDR)	Monitors endpoints, generates security alerts.
Attacking Tools	Atomic Red Team + Metasploit	Simulates MITRE ATT&CK techniques.
CTI Resource	MITREtrieval	Provide lifecycles for attacking
Python Libraries	Pytorch	Implementing of RL algorithm
	Matplotlib	Visualizes metrics and results.
	Seaborn	Creates statistical and correlation plots.

Table. Open-Source Tools and Libraries

Implementation – System Architecture

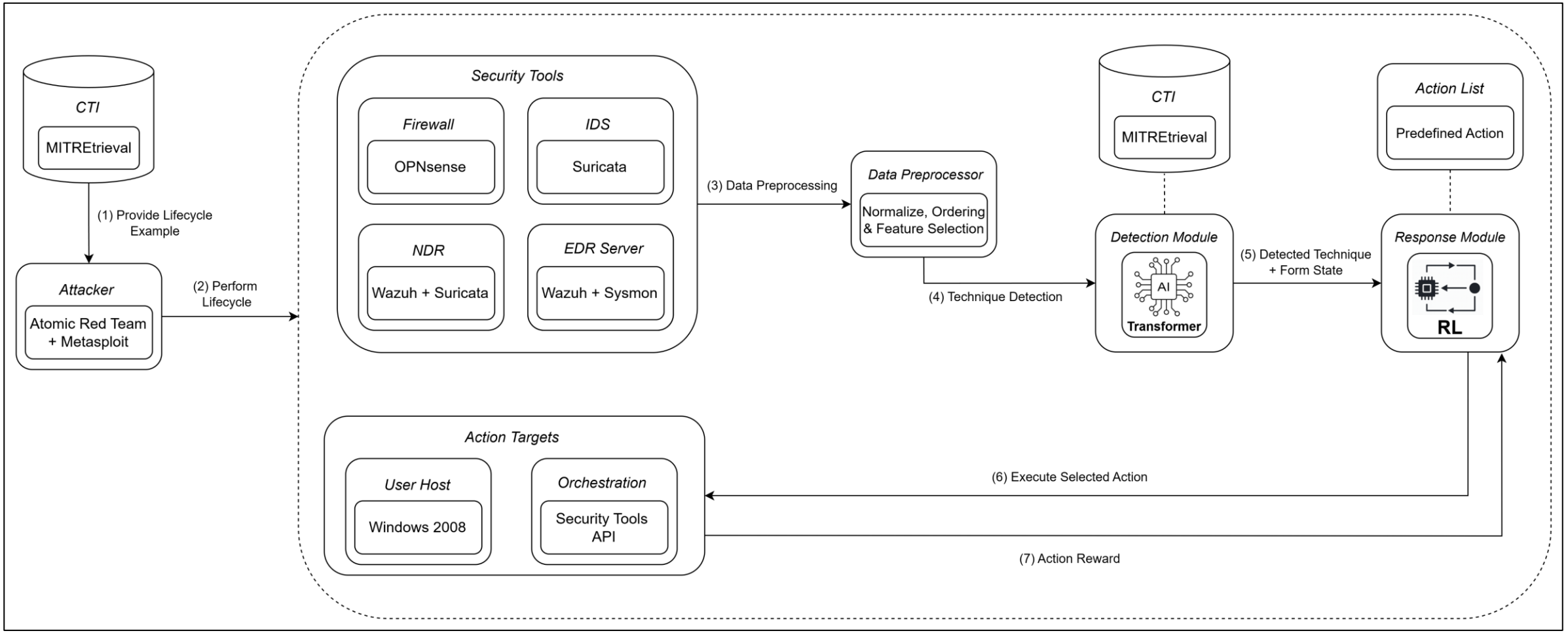


Figure. System Architecture

Implementation – Lifecycle Distribution

- Lifecycle Source:
  - Lifecycles are built from CTI reports using MITREtrieval
  - It extracts MITRE ATT&CK techniques from unstructured text
- Lifecycle Length Distribution:
  - Most lifecycles contain 5–10 techniques
  - Average lifecycle length: ~8.6 techniques
  - Maximum complexity: up to 19 techniques

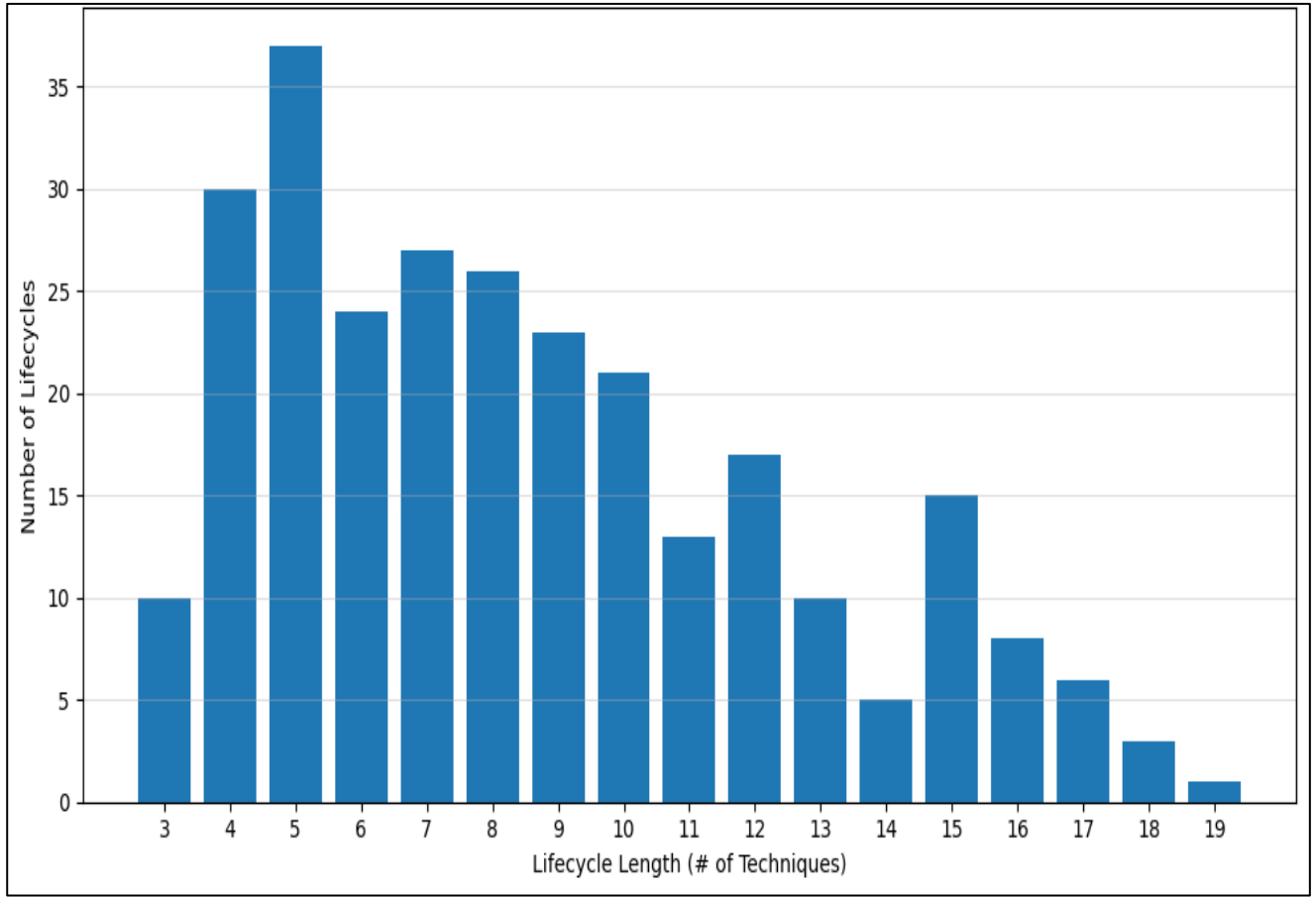


Figure. Lifecycle Distribution

Implementation – Technique Distribution Across Tactics and IoC Types

- Implemented Techniques in the Testbed
  - This figure shows the implemented techniques grouped by MITRE ATT&CK tactic.
  - In total, the testbed includes 91 unique techniques.
  - Top 3 tactics: Discovery (20), Defense Evasion (17), Command and Control (13).

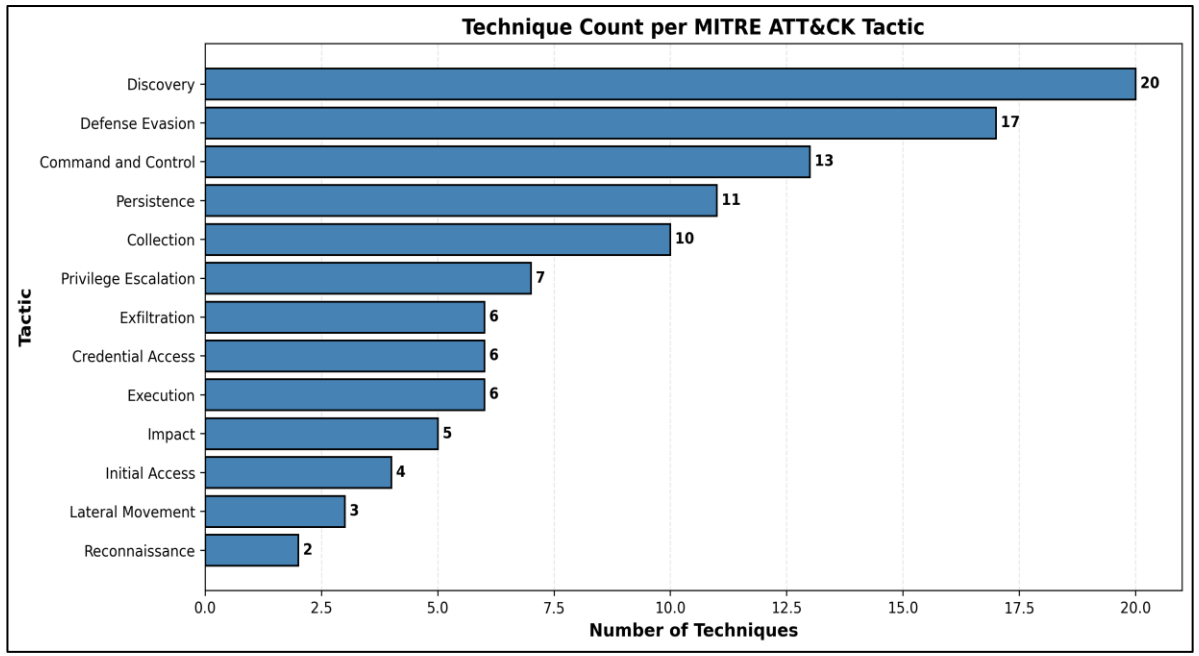


Figure. Technique Count per MITRE ATT&CK Tactic

- Technique Mapping by IoC Type
  - IoC represents the main compromise artifact or evidence left by a technique.
  - The same 91 techniques are grouped by IoC type.
  - Top 3 IoC types: File (51), Process (41), IP (32).

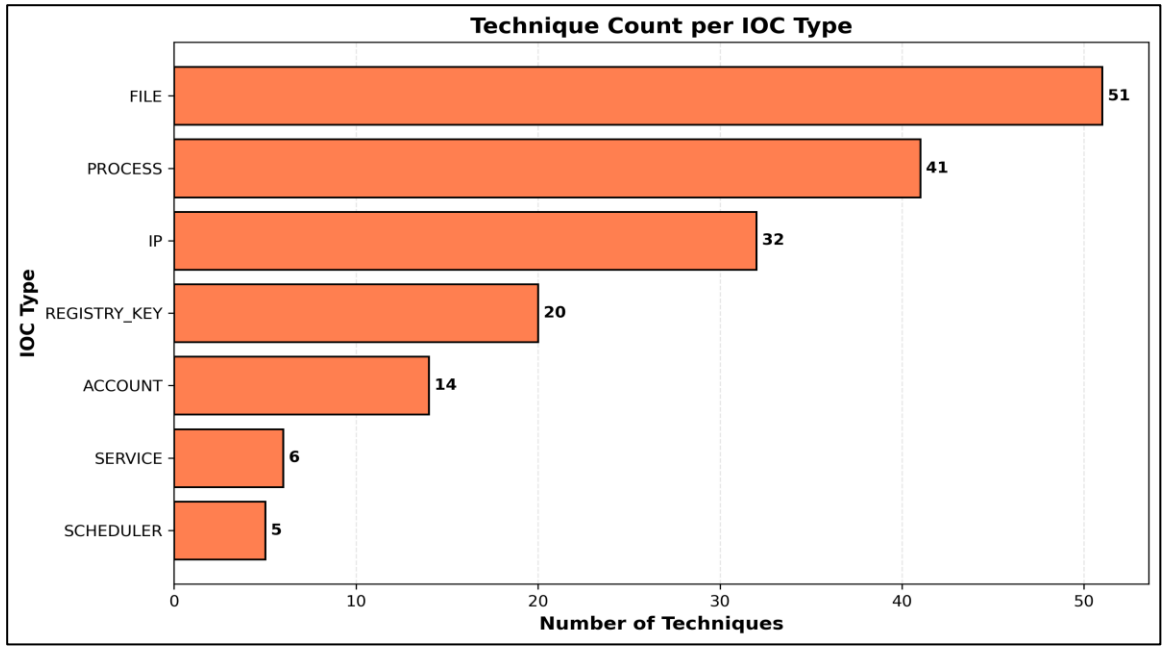


Figure. Technique Count per IoC Type

Implementation - Sighting Features

- Sighting Features
  - Selected key features from security tools
  - Include both network and host information
- Why This Feature
  - Network features
    - Capture communication behavior
    - Detect C2, scanning, and exfiltration
  - Host features
    - Capture system-level actions
    - Detect execution, persistence, and privilege abuse
  - Combined (Network & Host)
    - Provide a complete attack view
    - Improve technique detection

Feature Type	Security Tools	Sighting Features	Description
Network	Firewall	proto	Network protocol used
		dst_port	Destination port of the connection
	IDS	proto	Network protocol used
		dst_port	Destination port of the connection
	NDR	signature	Detected attack pattern
		severity	Threat severity level
Host	EDR	rule_desc	Detection rule triggered
		command_line	Executed command on the host

Figure. Sighting Features

Implementation - Detection Model Configuration and Sightings Statistics

- **Sighting Statistics**
  - Total collected alerts: **13,663**
  - Average alerts per window: **2.13**
  - Alert range per window: **1 to 8**
- **Alert Count by Security Tool**
  - **EDR:** 10,822
  - **IDS:** 1,521
  - **Firewall:** 828
  - **NDR:** 492

Item	Transformer	LLM
Train ratio	80% train / 20% test	
Model candidates	DistilRoBERTa, SecureBERT, SecRoBERTa	LLaMA 3 Instruct, Gemma 3, Foundation-Sec
Max sequence length	512	
Number of epochs	10	5
Learning rate	2e-5	2e-4
Gradient accumulation	2	32

Figure. LLM & Transformer Model Configuration

Implementation - Example of Sighting Input

- Input: multi-source alert evidence within a time window
- Output: detected target technique

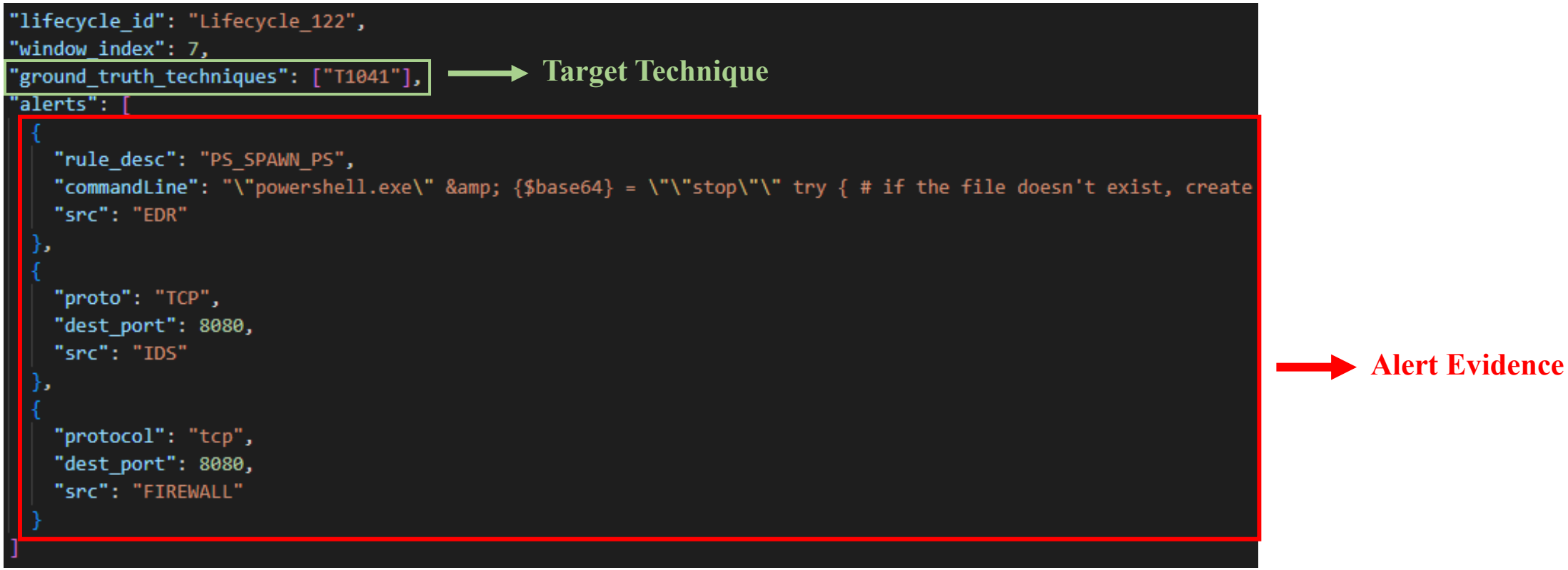


Figure. Example of Detection Input

Implementation – RL State (Current Stage)

- Why
  - Guides response strategy based on attack progression
  - Early stage → prefer low-impact actions (block lightly)
  - Late stage → allow stronger actions (isolate, shutdown, aggressive remediation)
- Define Current Stage (2 configuration comparison)
  - By Technique Progress
    - Early: 1–3
    - Medium: 4–6
    - Late: ≥7
  - By Tactic
    - Early: Reconnaissance, Initial Access, Execution
    - Medium: Persistence, Privilege Escalation, Defense Evasion, Discovery, Command & Control
    - Late: Credential Access, Lateral Movement, Collection, Exfiltration, Impact

Example Sequence	Count	Stage
T1595 Active Scanning T1190 Exploit Public-Facing Application T1078 Valid Accounts	3	Early
T1595 Active Scanning T1190 Exploit Public-Facing Application T1078 Valid Accounts T1059 Command and Scripting Interpreter T1083 File and Directory Discovery	5	Medium
T1595 Active Scanning T1190 Exploit Public-Facing Application T1078 Valid Accounts T1059 Command and Scripting Interpreter T1083 File and Directory Discovery T1021 Remote Services T1003 OS Credential Dumping T1041 Exfiltration Over C2 Channel	8	Late

Table. Example Current Stage Calculation by Technique Progress

Implementation – RL State (System Artifact)

- Why
  - Captures remaining compromise evidence for targeted recovery
  - Defined from behaviors observed in Atomic Red Team and Metasploit attacks
- Field on System Artifact
  - Network State: Captures ongoing attacker connections
  - Process State: Detects suspicious running processes
  - Account State: Shows compromised or abused accounts
  - Scheduler State: Tracks persistence via scheduled tasks
  - Service State: Identifies abused or created services
  - File State: Captures malicious files left on the host
  - Registry State: Detects persistence or system tampering

Artifact Field	Example Fields Inside
Process State	suspicious_processes
Network State	active_connections
Account State	account_artifacts session_artifacts
Scheduler State	scheduler_artifacts
Service State	service_artifacts
File State	file_artifacts modification_artifacts deleted_or_wiped_targets
Registry State	Registry setting being monitored Example: UseLogonCredential 0 = normal, 1 = unsafe (credentials stored in memory)

Table. RL System Artifact State Representation

Implementation – RL State (System Status)

- System Status: Indicates whether the host is in a NORMAL or COMPROMISED state
- How to Decide:
  - Compare the current system state with the baseline normal state
  - Normal: all related artifacts are removed, cleared, or restored to baseline values
  - Compromised: any related artifact still exists

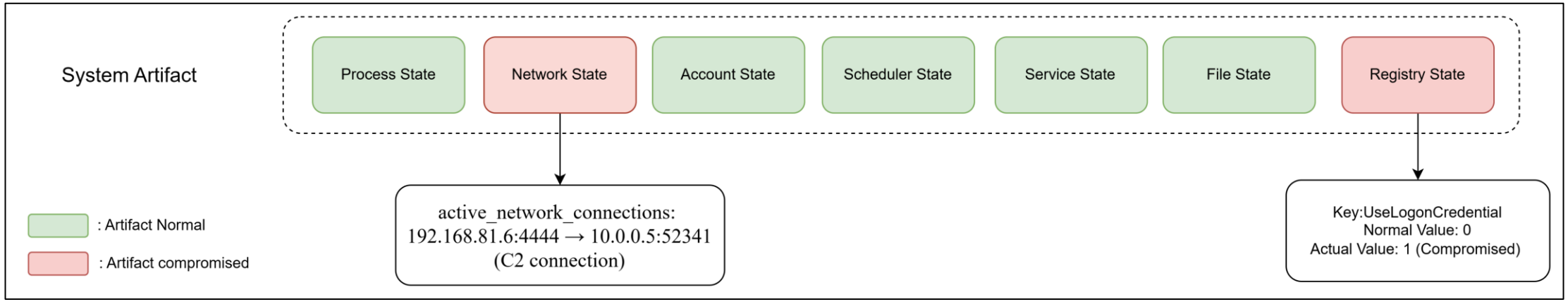


Figure. System Status Determination from Artifact States

Implementation – Response Model Configuration

Item	Setting
RL algorithm	DDQN (Double Deep Q-Network)
Training episodes	12,000
Estimated timesteps	240,000
Max steps per episode	20
Learning rate	5e-5
Batch size	256
Exploration fraction	0.15
Final exploration epsilon	0.02
Discount factor	0.99
Gradient steps	4
Network architecture	[256, 256]

Table. RL model configuration

Item	Setting
Backend/API	OpenRouter
Default models	• openai/gpt-4.1-mini • google/gemini-2.5-flash
Timeout	120 seconds

Table. LLM response model configuration

Implementation – LLM Response Prompt Design

- The LLM prompt $Prompt^{(l)}$ follows the same logic as the RL response design.
- The prompt includes the key state information used in RL:
  - current attack stage
  - system artifacts
  - system status
  - available response actions
- The prompt also follows the same response principle:
  - prefer low-impact actions in the early stage
  - allow stronger actions in the late stage
  - recover the system while avoiding unnecessary disruption
- This makes the LLM baseline closer to the RL policy structure, leading to a fairer comparison.

```
You are a cybersecurity response agent.

Current stage: Late
System status: COMPROMISED
Artifacts:
- suspicious process detected
- malicious registry key exists
- active outbound connection to suspicious IP

Available actions:
- Process cleanup
- Registry cleanup
- Reset TCP connection
- Isolate endpoint
- Shutdown workstation

Choose the best action to stop the attack and recover the system.
Prefer lower-impact actions when possible, but use stronger actions
if the attack is severe
```

Figure. Example of LLM Response Prompt

Evaluation

1. Technique Detection: Detect Attacker Movement
   - Metric: F1-score, Accuracy, recall and inference time
   - Measure: Compare detected technique against ground-truth technique and evaluate LLMs and Transformers based on detection performance and latency
2. Response Decision: Action Selection
   - Metric: recovery success rate, lifecycle stopping position, and stage-aware action behavior
   - Measure: Evaluate whether the RL agent restores the system and stops the attack lifecycle earlier across different stage designs
3. Response Model Comparison: LLM vs RL
   - Metrics: recovery success rate, lifecycle stopping position, and stage-aware action behavior
   - Measure: Compare RL and prompt-based LLM response across stage designs in recovery consistency and early lifecycle stopping
4. Architecture Comparison: Transformer+RL vs. LLM Detection–Response
   - Metrics: Detection latency, response latency, and end-to-end latency
   - Measure: Compare the runtime efficiency of Transformer+RL and LLM-based detection–response pipelines

Evaluation 1 – Technique Detection: Detect Attacker Movement
Transformer: Similar Accuracy with LLM, Lower Latency

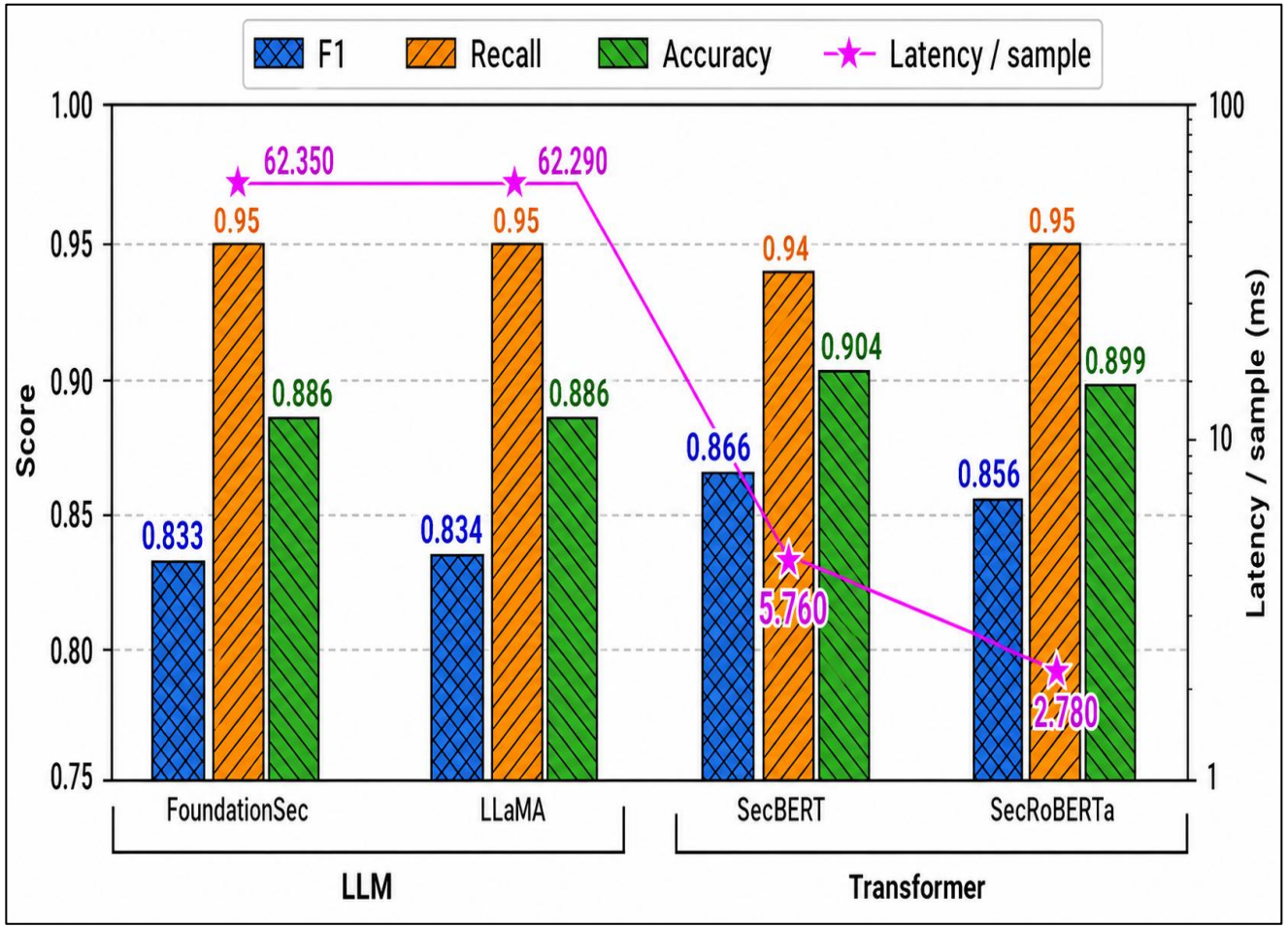


Figure. LLM vs Transformer Performance in Technique Detection

- **Observation & Insight:**
  - **F1:** Transformer higher, **0.856–0.866** vs **LLM 0.833–0.834** → better fit for the limited labeled dataset
  - **Latency gap:** Transformer much faster, **2.78–5.76 ms** vs **LLM 62.29–62.35 ms** → more practical for real-time SOC detection

Evaluation 2 – Response Decision: Action Selection
RL Learns Stable Recovery Across All Settings

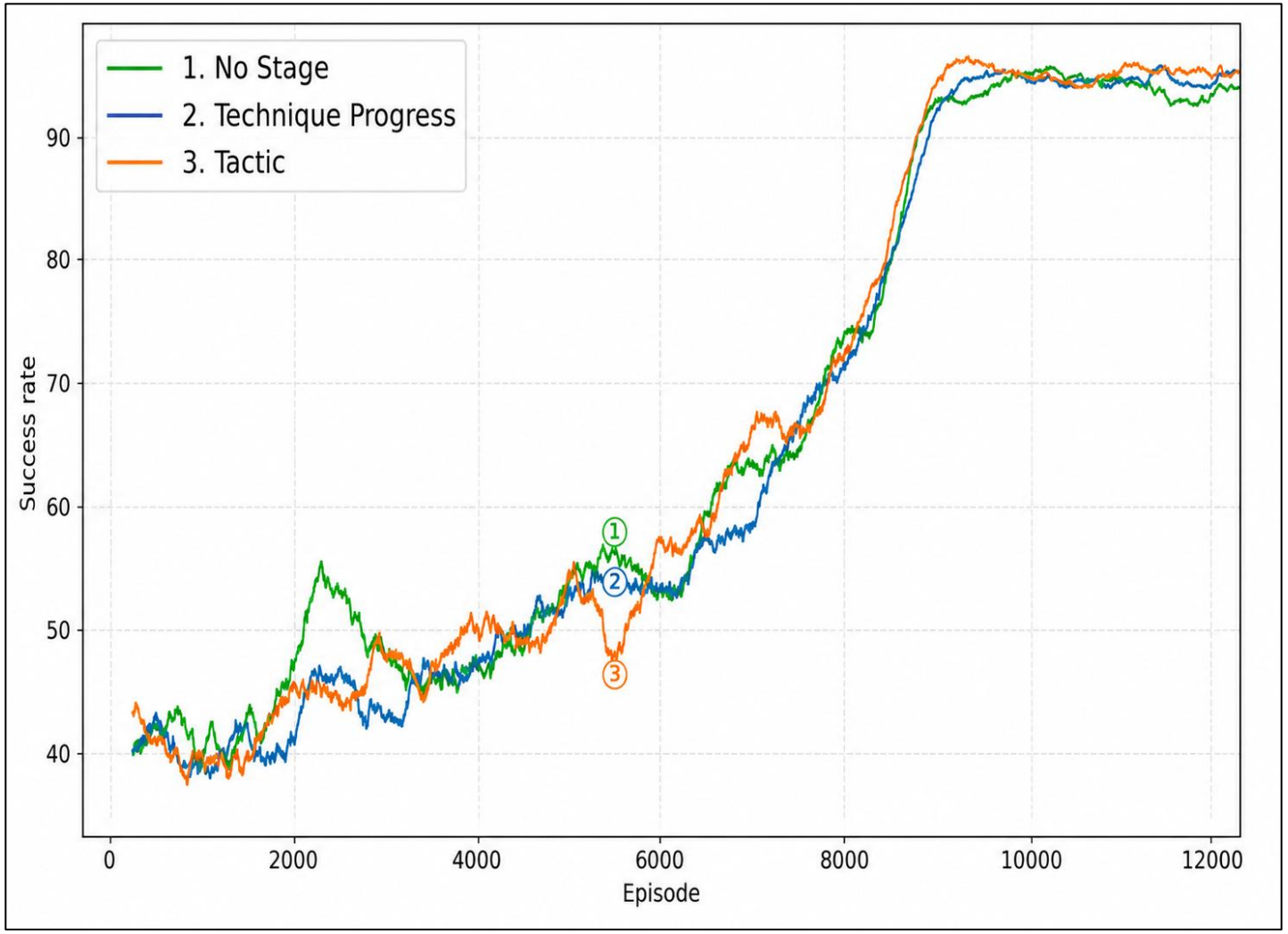


Figure. RL Learning Curve: System Recovery Success Rate per Episode

- **Observation & Insight:**
  - **Initial stage: 40%–50% success** → weak recovery policy
  - **After training:** above **90% after ~9,000 episodes** → stable recovery policy across settings

Evaluation 2 – Response Decision: Action Selection
Stage-Aware RL Adapts Action Strength across Stage

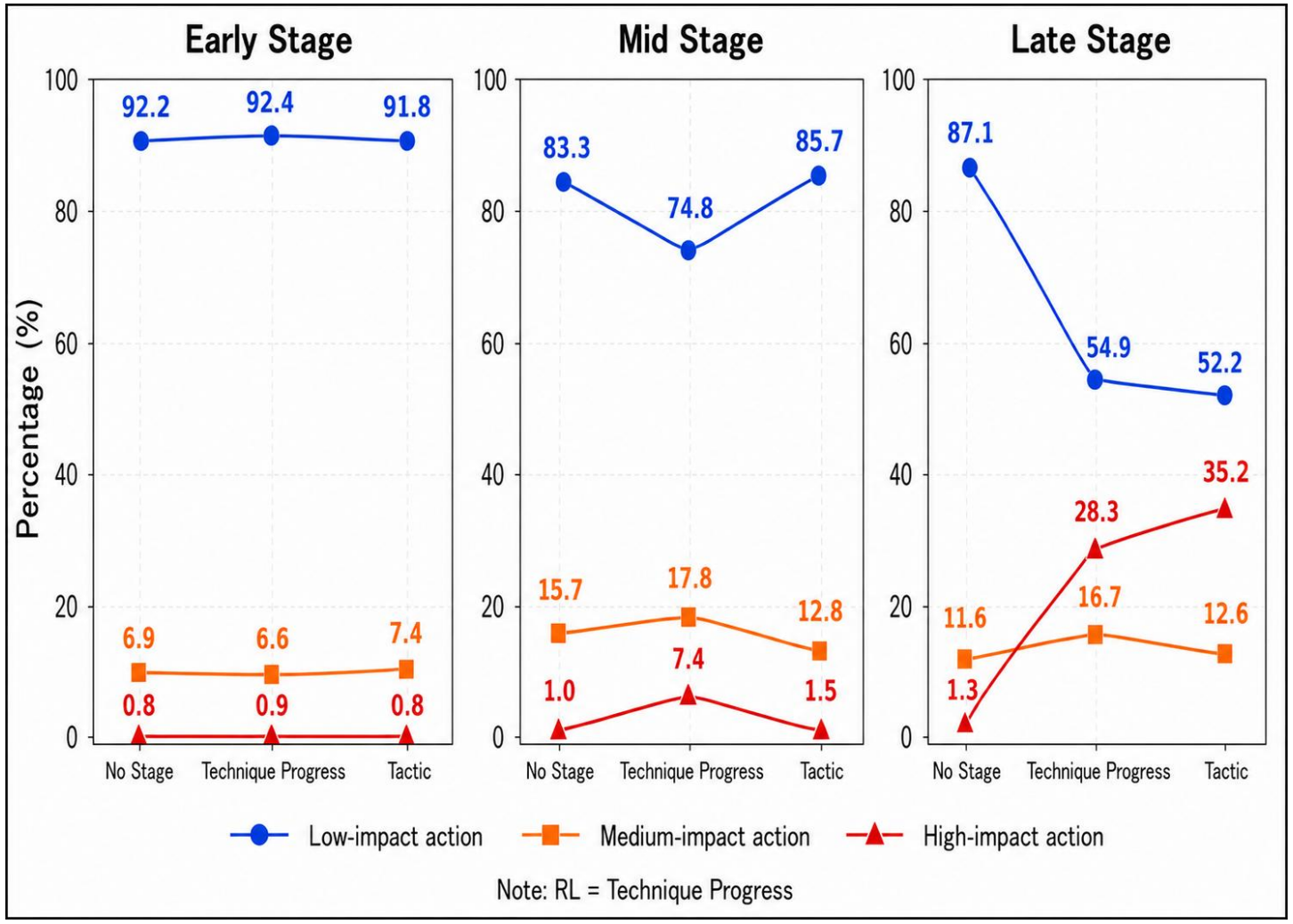


Figure. RL Action Selection Distribution Across Episodes

- **Observation & Insight:**
  - **Early stage:** low-impact dominates, **91.8%–92.4%** → conservative response before escalation
  - **Late stage:** Technique Progress / Tactic raise high-impact to **28.3%–28.8%** vs No Stage **1.3%** → stronger stage-aware mitigation

Evaluation 2 – Response Decision: Action Selection

Stage-Aware RL Stops Lifecycle Earlier

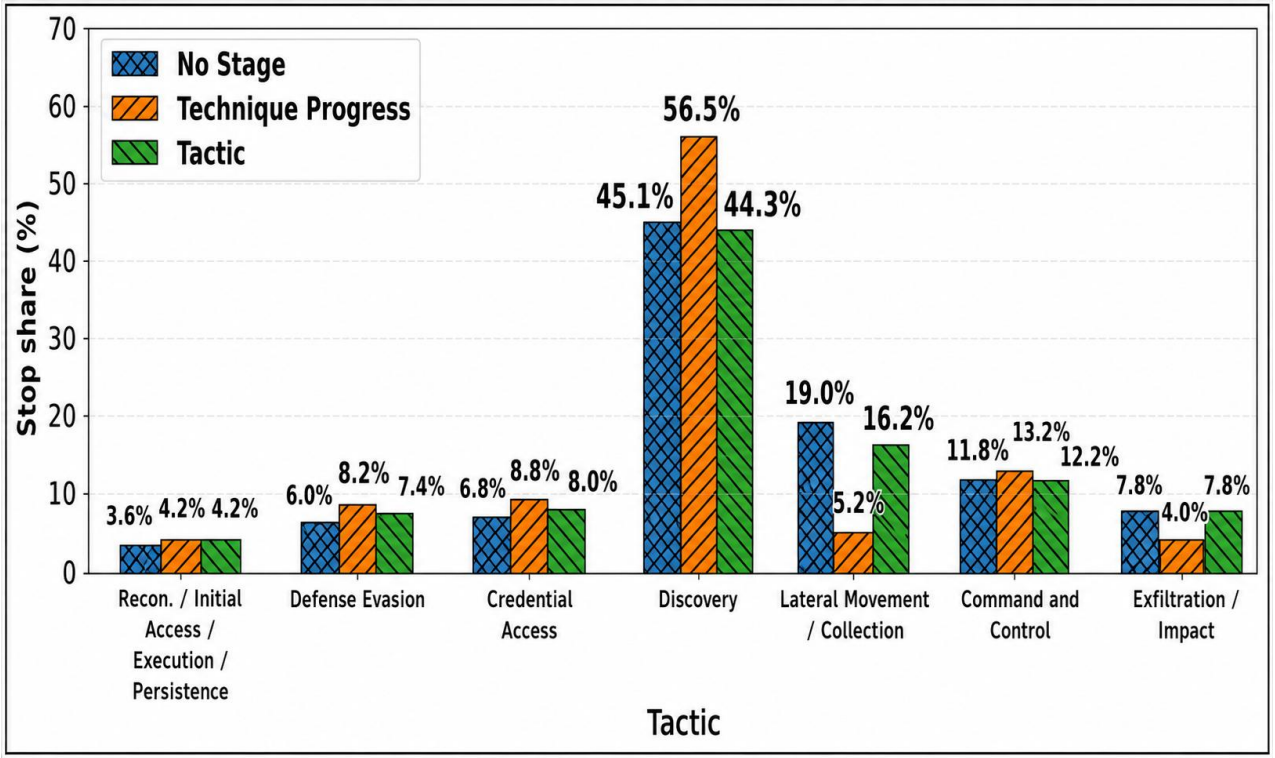


Figure. Lifecycle Stopping Distribution by Tactic under Different RL Stage Designs

Order	Tactic Group	No Stage Cumulative	Technique Progress Cumulative	Tactic Cumulative
1	Recon. / Initial Access / Execution / Persistence	3.6%	4.2%	4.2%
2	Defense Evasion	9.6%	12.4%	11.6%
3	Credential Access	16.4%	21.2%	19.6%
4	Discovery	61.5%	**77.7%**	63.9%
5	Lateral Movement / Collection	80.5%	82.9%	80.1%
6	Command and Control	92.3%	96.1%	92.3%
7	Exfiltration / Impact	100%	100%	100%

Table. Cumulative Lifecycle Stopping Share by Tactic

- **Observation & Insight:**
  - **Discovery stop:** Technique Progress highest, **56.5%** vs No Stage **45.1%** and Tactic **44.3%** → more lifecycles stopped at Discovery
  - **Cumulative stop:** Technique Progress highest across tactics → strongest lifecycle stopping overall

Evaluation 3 – Response Model Comparison: LLM vs RL
RL Recovers Better than LLM-Based Response

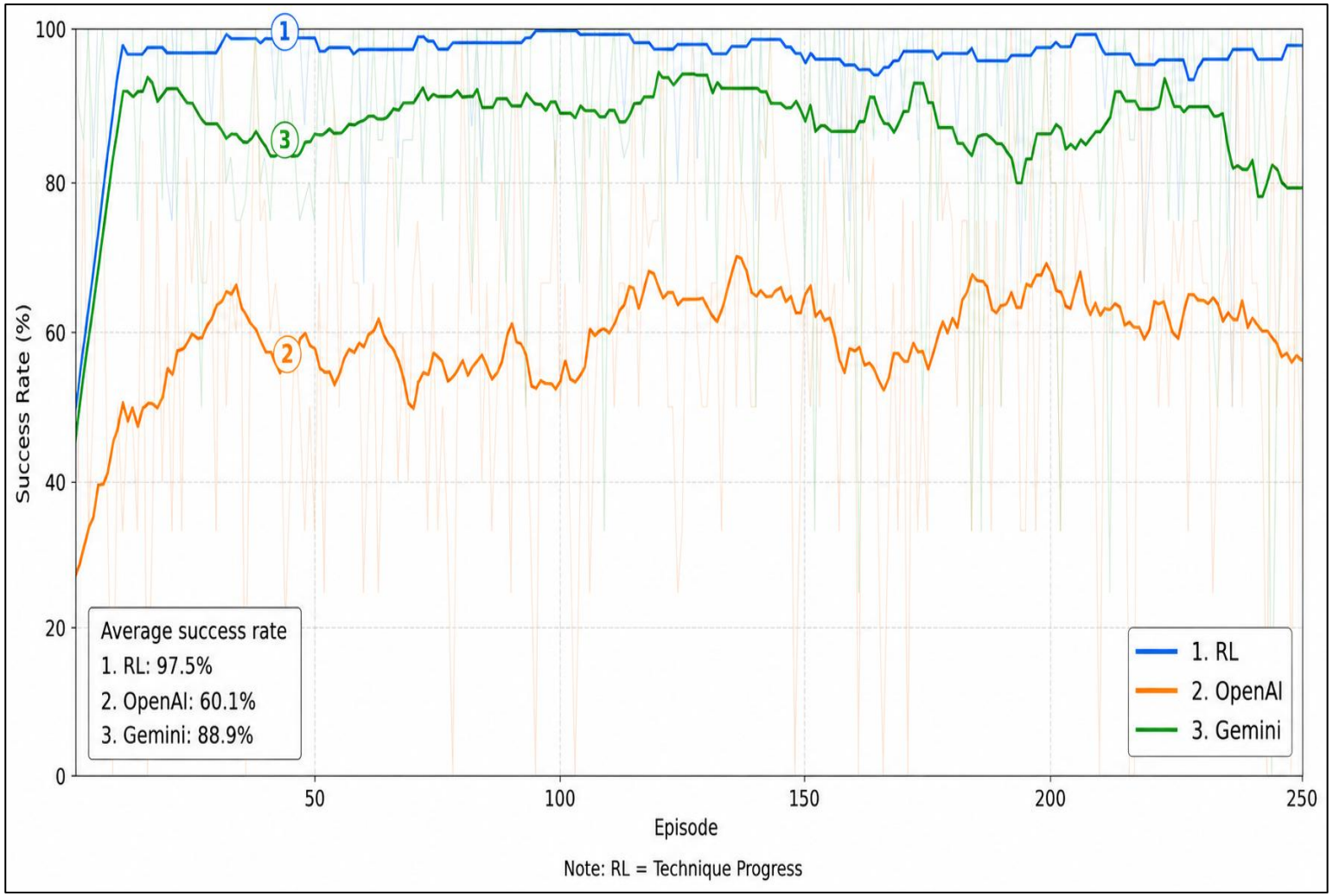
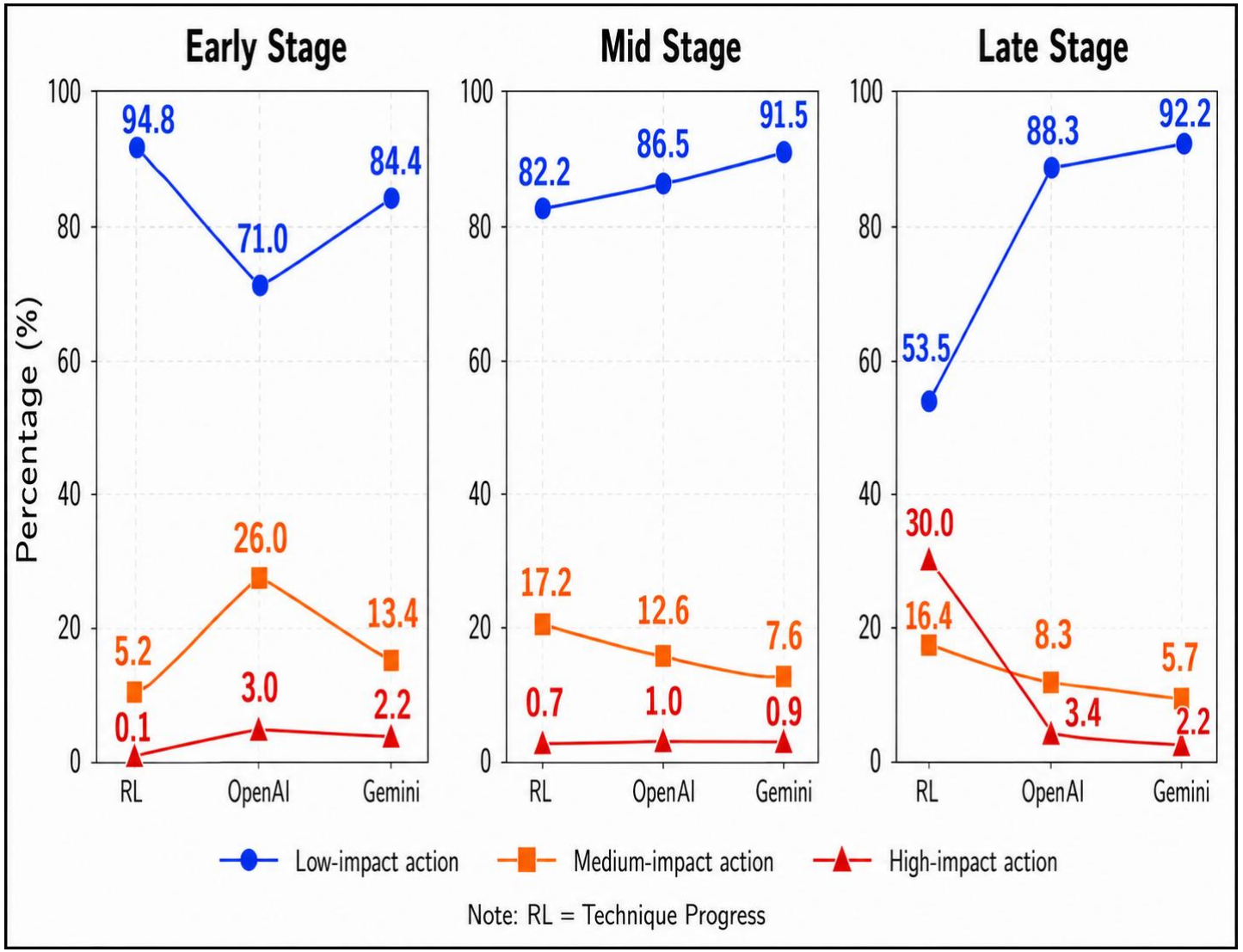


Figure. System Recovery Success Rate of RL and LLM Response Models

- **Observation & Insight:**
  - **Recovery success:** RL mostly above **95%** vs Gemini **80%–90%** and OpenAI **50%–70%** → stronger response action selection
  - **LLM limitation:** prompt-based response is less consistent → fine-tuning may be needed for response quality

Evaluation 3 – Response Model Comparison: LLM vs RL
Stage-Aware RL Adapts Better than LLM



- **Observation & Insight:**
  - **Early stage:** RL mostly **low-impact 94.8%**; LLMs use more **medium-impact 26.0% & 13.4%** → RL responds more conservatively early
  - **Late stage:** RL increases **high-impact actions to 30.0%** vs LLM **3.4%** & **2.2%** → stage-aware RL adapts better to severe attack stages

Figure. Action Selection Distribution: RL vs LLM Response Models

Evaluation 3 – Response Model Comparison: LLM vs RL
RL Stops Lifecycles Earlier than LLM-Based Response

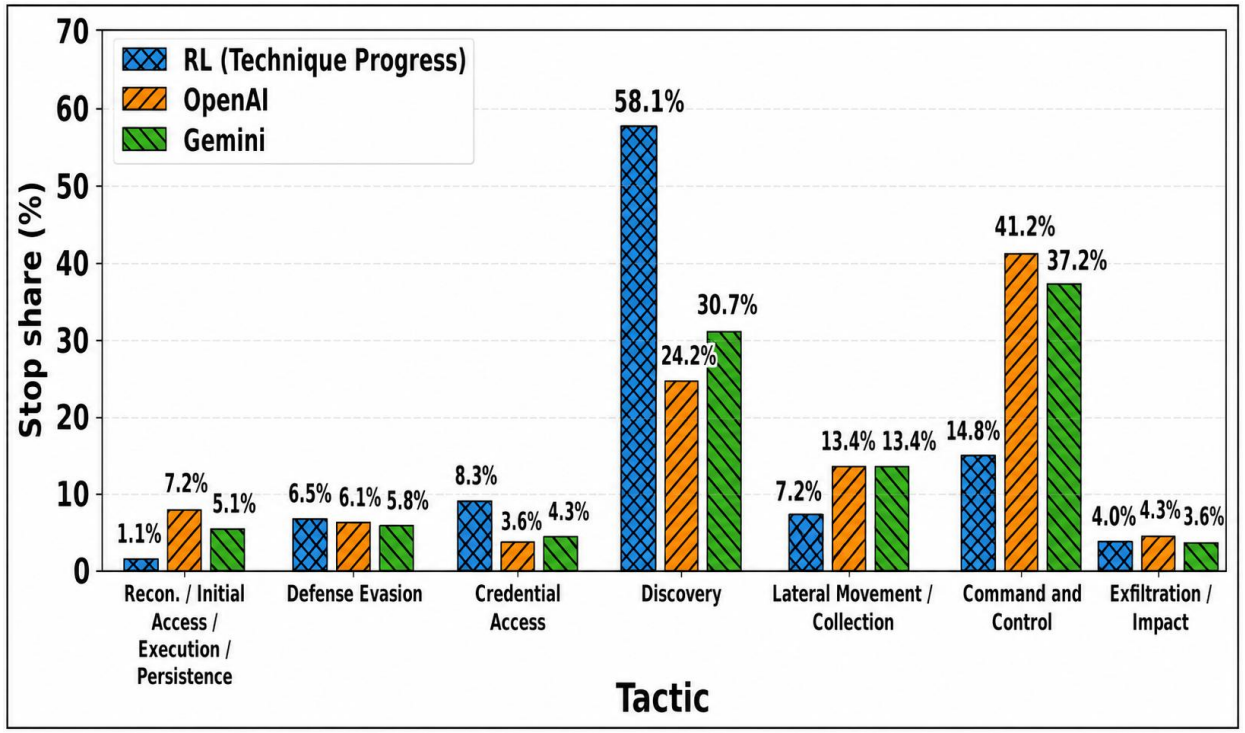


Figure. Lifecycle Stopping Distribution by Tactic between RL and LLM

Order	Tactic Group	RL Cumulative	OpenAI Cumulative	Gemini Cumulative
1	Recon. / Initial Access / Execution / Persistence	1.1%	7.2%	5.1%
2	Defense Evasion	7.6%	13.3%	10.9%
3	Credential Access	15.9%	16.9%	15.2%
4	Discovery	74.0%	41.1%	45.9%
5	Lateral Movement / Collection	81.2%	54.5%	59.3%
6	Command and Control	96.0%	95.7%	96.5%
7	Exfiltration / Impact	100%	100%	100%

Table. Cumulative Lifecycle Stopping Share by Tactic

- **Observation & Insight:**
  - **Discovery stop:** RL highest, **58.1%** vs OpenAI **24.2%** and Gemini **30.7%** → more lifecycles stopped at Discovery
  - **Cumulative stop:** RL leads by **20%–30%** at **Discovery** and **Lateral Movement / Collection** → stronger early lifecycle stopping

Evaluation 4 – Architecture Comparison: Transformer+RL vs. LLM Detection–Response
Transformer+RL Achieves Much Lower End-to-End Latency

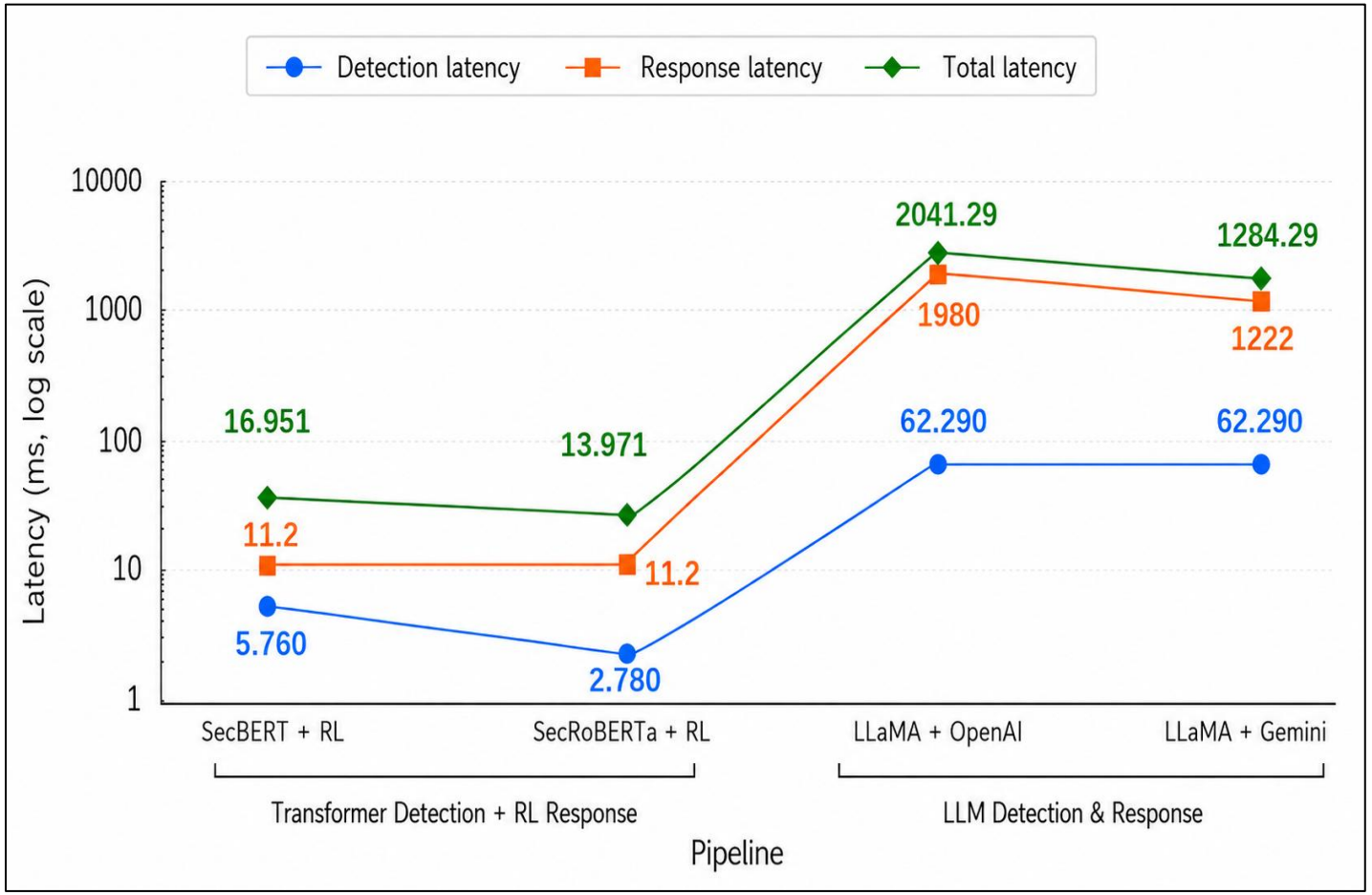


Figure. End-to-End Comparison of Transformer+RL and LLM-Based Detection–Response Pipelines

- **Observation & Insight:**
  - **Total latency:** Transformer+RL only **13.97–16.95 ms** vs LLM pipelines **1284.29–2041.29 ms** → much better for real-time SOC operation
  - **Main bottleneck:** LLM response latency **1222–1980 ms** vs RL response **11.2 ms** → LLM-based response limits end-to-end speed

Schedules

Item Name	2025					2026				
	8	9	10	11	12	1	2	3	4	5
Thesis Survey										
Testbed Setup										
Attack simulation & Data Collection										
Detection (Transformer)										
Detection (LLM)										
Response Single Agent										
Response Dual Agent										
System Evaluation & Comparison										
Thesis Writing										

Table 8. Thesis Schedules

References

[1] F. J. Aparicio-Navarro, K. G. Kyriakopoulos, I. Ghafir, S. Lambotharan and J. A. Chambers, "Multi-Stage Attack Detection Using Contextual Information," MILCOM 2018 - 2018 IEEE Military Communications Conference (MILCOM), Los Angeles, CA, USA, 2018, pp. 1-9, doi: 10.1109/MILCOM.2018.8599708.

[2] Li, He. "Research on decision optimization of network security expert system based on multi-source heterogeneous data." J. COMBIN. MATH. COMBIN. COMPUT 127 (2025): 8683-8700.

[3] Shahroz Tariq, Mohan Baruwal Chhetri, Surya Nepal, and Cecile Paris. 2025. Alert Fatigue in Security Operations Centres: Research Challenges and Opportunities. ACM Comput. Surv. 57, 9, Article 224 (September 2025), 38 pages. https://doi.org/10.1145/3723158

[4] Y.-D. Lin, C.-Y. Chang, D. Sudyana, Y.-C. Lai, R.-H. Hwang, P.-C. Lin, E. H.-K. Wu, and W.-B. Lee, “LLM-driven alert correlation and lifecycle prediction with threat intelligence for attack forensics,” 2026.

[5] A. R. Muhammad, P. Sukarno, and A. A. Wardana, “Integrated Security Information and Event Management (SIEM) with Intrusion Detection System (IDS) for Live Analysis based on Machine Learning,” Procedia Computer Science, vol. 217, pp. 1406–1415, 2023, doi: 10.1016/j.procs.2022.12.339.

[6] K. Malialis and D. Kudenko, “Distributed response to network intrusions using multiagent reinforcement learning,” Engineering Applications of Artificial Intelligence, vol. 41, pp. 270–284, 2015, doi: 10.1016/j.engappai.2015.01.013.

[7] K. Hughes, K. McLaughlin, and S. Sezer, “A Model-Free Approach to Intrusion Response Systems,” Journal of Information Security and Applications, vol. 66, p. 103150, 2022, doi: 10.1016/j.jisa.2022.103150.

[8] I. Ismail, R. Kurnia, Z. A. Brata, G. A. Nelistiani, S. Heo, H. Kim, and H. Kim, “Toward Robust Security Orchestration and Automated Response in Security Operations Centers with a Hyper-Automation Approach Using Agentic Artificial Intelligence,” Information, vol. 16, no. 5, p. 365, 2025, doi: 10.3390/info16050365.

[9] IBM, “What Is a Security Operations Center (SOC)?,” 2025. [Online]. Available: https://www.ibm.com/think/topics/security-operations-center

[10] The MITRE Corporation, “MITRE ATT&CK®,” 2025. [Online]. Available: https://attack.mitre.org/

References

[11] Palo Alto Networks, “What Is Cyber Threat Intelligence (CTI)?,” 2025. [Online]. Available: https://www.paloaltonetworks.com/cyberpedia/what-is-cyberthreat-intelligence-cti

[12] Huang, Y. T., Vaitheeshwari, R., Chen, M. C., Lin, Y. D., Hwang, R. H., Lin, P. C., Lai, Y. C., Wu, E. H. K., Chen, C. H., Liao, Z. J., & Chen, C. K. 2024. MITREtrieval: Retrieving MITRE Techniques From Unstructured Threat Reports by Fusion of Deep Learning and Ontology. IEEE Transactions on Network and Service Management, 21(4), 4871-4887. https://doi.org/10.1109/TNSM.2024.3401200

[13] IBM, “What Is a Transformer Model?,” 2025. [Online]. Available: https://www.ibm.com/think/topics/transformer-model

[14] IBM, “What Are Large Language Models?,” 2025. [Online]. Available: https://www.ibm.com/think/topics/large-language-models

[15] IBM, “What Is Fine-Tuning?,” 2025. [Online]. Available: https://www.ibm.com/think/topics/fine-tuning

[16] IBM, “What Is Reinforcement Learning?,” 2025. [Online]. Available: https://www.ibm.com/think/topics/reinforcement-learning

[17] IBM, “What Are AI Agents?,” 2026. [Online]. Available: https://www.ibm.com/think/topics/ai-agents

[18] J. Lee, J. Kim, J. Kim, and K. Han, “Cyber threat detection based on artificial neural networks using event profiles,” IEEE Access, vol. 7, pp. 165607–165626, 2019, doi: 10.1109/ACCESS.2019.2953095.

[19] K. Hughes, K. McLaughlin, and S. Sezer, “Dynamic Countermeasure Knowledge for Intrusion Response Systems,” 2020 31st Irish Signals and Systems Conference (ISSC), Letterkenny, Ireland, 2020, pp. 1–6, doi: 10.1109/ISSC49998.2020.9180198.

[20] E. Iturbe, A. Rego, O. Llorente-Vazquez, E. Rios, C. Dalamagkas, D. Merkouris, and N. Toledo, “Reinforcement Learning in action: Powering intelligent intrusion responses to advanced cyber threats in realistic scenarios,” Expert Systems With Applications, vol. 296, p. 129168, 2026, doi: 10.1016/j.eswa.2025.129168.

Appendix – Testbed Setup

Component / Role	Platform	Data flow
Attacker	Kali Linux	1. Atomic Red Team + Metasploit executed on attacker 2. Delivered to victim via SSH
SOC server	Ubuntu 20.04 LTS (Wazuh)	1. Receives endpoint telemetery 2. Maps to alerts using Wazuh rules 3. Stores alerts under /var/log/ossec/
Victim / EDR	Windows Server 2008 R2 & Sysmon	1. Sysmon EDR telemetry (collected by Wazuh agent) 2. Forwarded to SOC server
NDR	Suricata	1. Network alert events (mapped by Suricata rules) 2. Stored under fast.log
IDS	Suricata	Detection events (eve.json)
Firewall	OPNsense	Firewall filter logs (/var/log/filter)

Table 7. Testbed Setup

Appendix 1 – Response Decision: Action Selection
Reward Collected per Episode

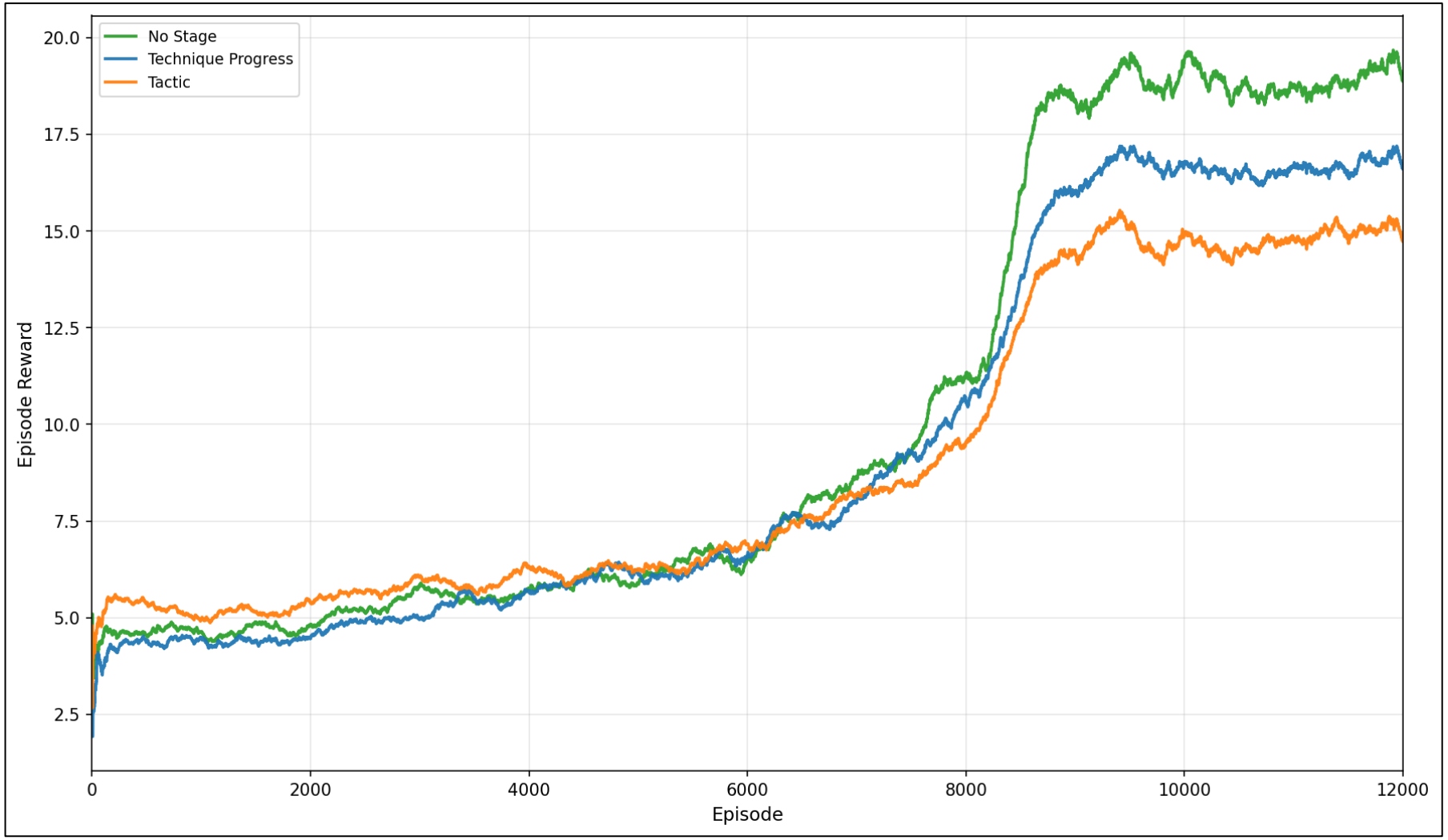


Figure. RL Learning Curve: Reward Progression per Episode

Appendix 2 – Response Decision: Action Selection

Action Distribution

Action	1-1000	1001-2000	2001-3000	3001-4000	4001-5000	5001-6000	6001-7000	7001-8000	8001-9000	9001-10000	10001-11000	11001-12000
Force sign out	1.60	1.29	1.11	1.34	1.81	1.50	0.92	0.56	0.78	0.85	0.87	0.46
Process cleanup	9.61	10.52	14.74	16.97	15.85	18.28	23.90	27.58	33.37	35.26	33.61	34.13
File cleanup	11.82	10.28	14.03	16.27	19.58	21.64	23.95	28.05	34.64	36.24	36.26	35.68
Registry cleanup	2.36	2.26	2.68	3.75	3.87	5.34	6.00	6.78	7.95	7.89	8.00	8.29
Service / Task cleanup	1.60	2.45	2.71	3.29	3.58	3.72	4.63	4.69	5.47	5.04	5.21	5.21
Reset TCP connection	4.58	4.40	3.28	3.05	5.56	4.57	4.98	2.92	3.53	3.61	3.82	4.45
Force password reset	1.14	1.32	1.57	1.86	1.61	1.57	1.58	2.05	2.75	1.86	2.19	1.70
Disable user account	1.83	0.90	1.00	1.13	0.83	0.92	1.17	1.07	0.57	0.91	0.81	1.22
Add IP to block list	4.46	5.22	5.17	4.90	4.01	4.47	4.75	6.71	6.04	5.99	5.65	5.34
Restrict command execution	12.70	12.00	10.53	9.59	8.48	8.00	6.15	4.06	1.50	0.50	1.61	1.57
Restrict write operations	13.96	13.60	11.46	11.76	10.12	9.60	7.54	6.12	1.26	0.68	0.90	0.95
Shutdown workstation	15.56	16.09	16.81	11.27	11.32	9.45	5.90	4.14	0.85	0.73	0.61	0.50
Isolate endpoint	18.76	19.67	14.92	14.83	13.38	10.93	8.52	5.27	1.30	0.45	0.47	0.48

Figure. RL No-Stage Action Distribution

Appendix 3 – Response Decision: Action Selection
Action Distribution

Action	1-1000	1001-2000	2001-3000	3001-4000	4001-5000	5001-6000	6001-7000	7001-8000	8001-9000	9001-10000	10001-11000	11001-12000
Force sign out	1.23	1.57	1.02	1.84	1.32	1.27	0.98	0.63	0.32	0.14	0.59	0.69
Process cleanup	10.23	10.45	13.85	17.81	18.12	18.23	22.73	25.98	29.61	31.03	29.81	29.92
File cleanup	11.07	12.17	12.76	15.41	18.87	21.10	22.35	28.34	32.57	34.10	34.22	33.91
Registry cleanup	2.34	1.84	2.88	3.65	4.03	5.07	5.52	6.62	7.35	7.48	7.75	8.07
Service / Task cleanup	2.66	2.00	2.38	3.44	4.19	3.87	4.72	4.97	6.08	5.76	5.97	5.86
Reset TCP connection	4.80	4.69	3.82	4.39	4.28	5.56	6.44	4.32	3.69	3.42	3.96	3.65
Force password reset	1.27	1.42	1.05	1.27	0.63	0.90	1.10	1.17	1.28	1.29	1.08	1.11
Disable user account	1.31	1.38	1.75	1.27	1.54	1.32	1.20	1.68	2.28	2.08	2.41	2.16
Add IP to block list	4.28	4.65	5.36	4.24	4.03	4.57	4.28	5.56	6.33	6.93	6.63	6.79
Restrict command execution	12.61	11.94	10.80	9.90	8.47	8.58	6.76	3.81	1.54	0.93	0.56	0.47
Restrict write operations	12.26	12.56	13.18	11.23	10.18	9.48	7.70	5.13	1.78	0.63	0.88	1.11
Shutdown workstation	15.15	15.21	14.45	10.88	10.46	9.41	6.92	6.11	5.39	5.75	5.66	5.75
Isolate endpoint	20.79	20.12	16.69	14.67	13.88	10.63	9.33	5.66	1.76	0.45	0.49	0.52

Figure. RL Technique Progress Action Distribution

Appendix 4 – Response Decision: Action Selection
Action Distribution

Action	1-1000	1001-2000	2001-3000	3001-4000	4001-5000	5001-6000	6001-7000	7001-8000	8001-9000	9001-10000	10001-11000	11001-12000
Force sign out	1.04	1.39	1.08	1.70	1.21	1.95	0.70	0.88	0.57	0.64	0.76	0.76
Process cleanup	10.89	10.61	13.65	16.03	16.11	17.94	22.15	25.84	32.53	33.91	32.68	32.58
File cleanup	11.16	10.46	13.61	17.22	20.03	21.62	24.68	28.77	34.13	35.33	35.07	34.64
Registry cleanup	2.19	2.16	2.89	3.86	3.58	4.00	5.46	6.44	7.48	7.32	7.60	7.91
Service / Task cleanup	1.73	2.16	2.52	3.47	3.44	4.09	5.01	5.28	5.83	5.40	5.59	5.57
Reset TCP connection	4.93	4.51	3.70	3.01	4.11	5.46	3.61	2.43	2.35	2.63	2.74	2.07
Force password reset	1.62	1.54	1.61	1.64	1.55	1.32	2.09	1.11	0.57	0.42	0.61	0.81
Disable user account	1.31	1.12	1.28	1.13	1.10	0.63	1.07	1.99	2.62	2.56	2.63	2.41
Add IP to block list	3.96	4.36	5.48	5.08	4.68	3.70	4.78	5.50	5.47	5.57	5.21	5.33
Restrict command execution	13.36	12.81	11.19	9.10	8.23	7.68	6.97	4.71	1.38	0.78	1.05	1.88
Restrict write operations	13.63	13.97	13.45	11.50	10.96	9.82	7.18	5.01	1.19	0.63	0.81	0.64
Shutdown workstation	15.32	15.66	14.55	12.02	12.00	10.36	8.25	6.13	3.98	3.96	4.22	4.37
Isolate endpoint	18.86	19.25	14.99	14.24	13.01	11.43	8.06	5.90	1.90	0.86	1.02	1.01

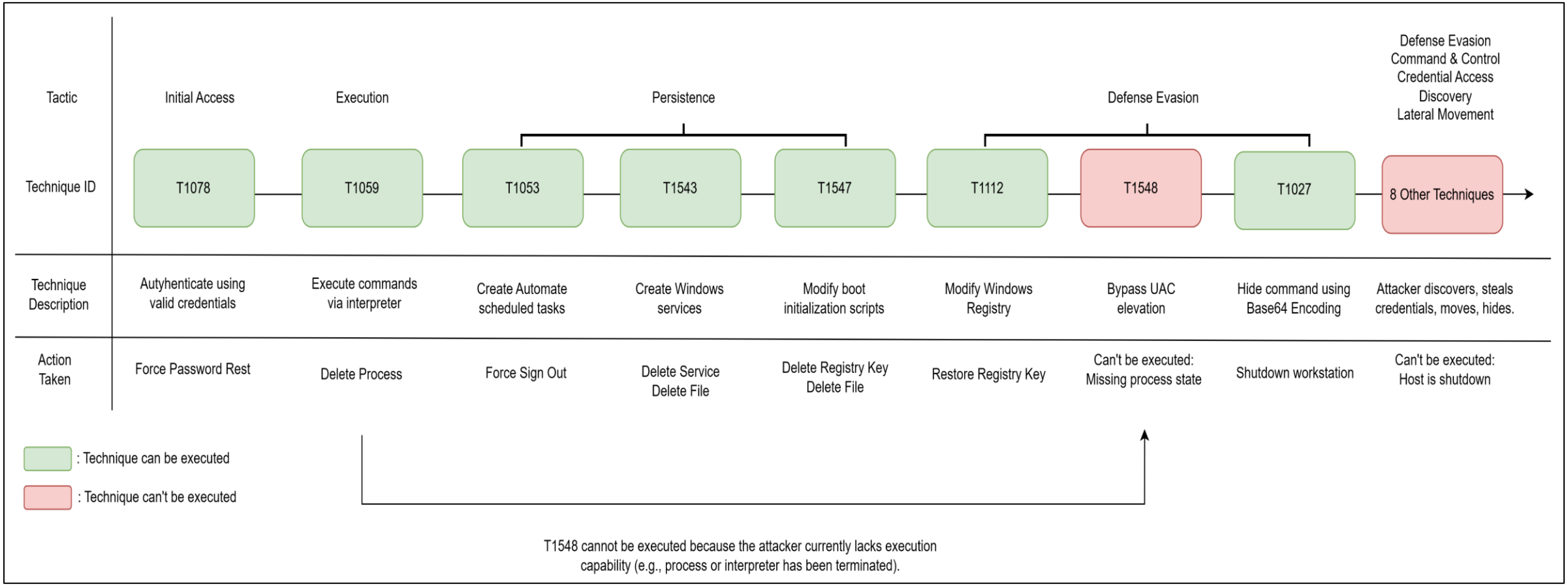
Figure. RL Tactic Action Distribution

Appendix 5 – Response Decision: Action Selection RL Qtable

Tactic	Force sign out	Process cleanup	File cleanup	Registry cleanup	Service / Task cleanup	Reset TCP connection	Force password reset	Disable user account	Add IP to block list	Restrict command execution	Restrict write operations	Shutdown workstation	Isolate endpoint
Collection	0.99	**2.13**	1.95	1.66	1.98	0.96	0.45	1.67	0.96	**1.99**	0.67	**-0.09**	-2.35
Command and Control	3.18	4.69	**4.97**	4.53	4.72	2.99	2.66	3.66	**4.22**	3.31	3.49	**3.74**	3.22
Credential Access	0.94	**1.93**	1.77	1.17	1.60	0.97	0.35	**1.45**	0.76	1.40	0.06	**-0.14**	-2.39
Defense Evasion	14.98	**16.93**	16.40	15.12	16.60	14.67	15.17	**15.39**	13.04	12.99	10.49	**5.53**	1.90
Discovery	3.16	4.03	4.17	3.77	**4.17**	2.86	2.91	**4.04**	2.69	3.73	2.87	**2.24**	0.06
Execution	15.43	17.15	16.92	16.18	**17.21**	15.98	15.09	**15.50**	13.49	11.88	10.76	**5.47**	2.33
Exfiltration	0.58	**2.18**	2.17	2.07	2.12	1.70	0.66	1.76	**1.90**	1.43	1.02	**-0.15**	-0.76
Impact	0.88	**2.51**	2.50	2.14	2.26	1.15	0.62	**2.12**	1.48	1.76	0.65	**-0.03**	-1.94
Initial Access	15.42	**17.52**	16.99	16.29	17.26	15.96	15.57	**15.75**	14.28	12.31	11.33	**5.67**	2.52
Lateral Movement	16.51	18.86	18.82	18.32	**19.15**	18.03	17.04	**17.09**	16.41	12.48	13.39	**6.09**	4.53
Persistence	14.98	**16.90**	16.49	15.68	16.74	15.07	15.03	**15.56**	13.50	12.63	10.91	**5.68**	2.22
Privilege Escalation	15.44	**17.37**	16.94	15.92	17.21	15.29	15.59	**16.07**	13.64	13.38	11.16	**5.79**	2.16
Reconnaissance	0.44	**2.09**	2.05	1.62	1.83	1.04	0.46	**1.56**	1.48	0.85	0.61	**0.05**	-1.06

Figure. RL Technique Progress Qtable

Appendix 6 – Response Decision: Action Selection
Example of Action Selection Affecting Technique Execution



Appendix 7 – Response Decision: Action Selection
Early vs Late Training Action Distribution Across Configurations

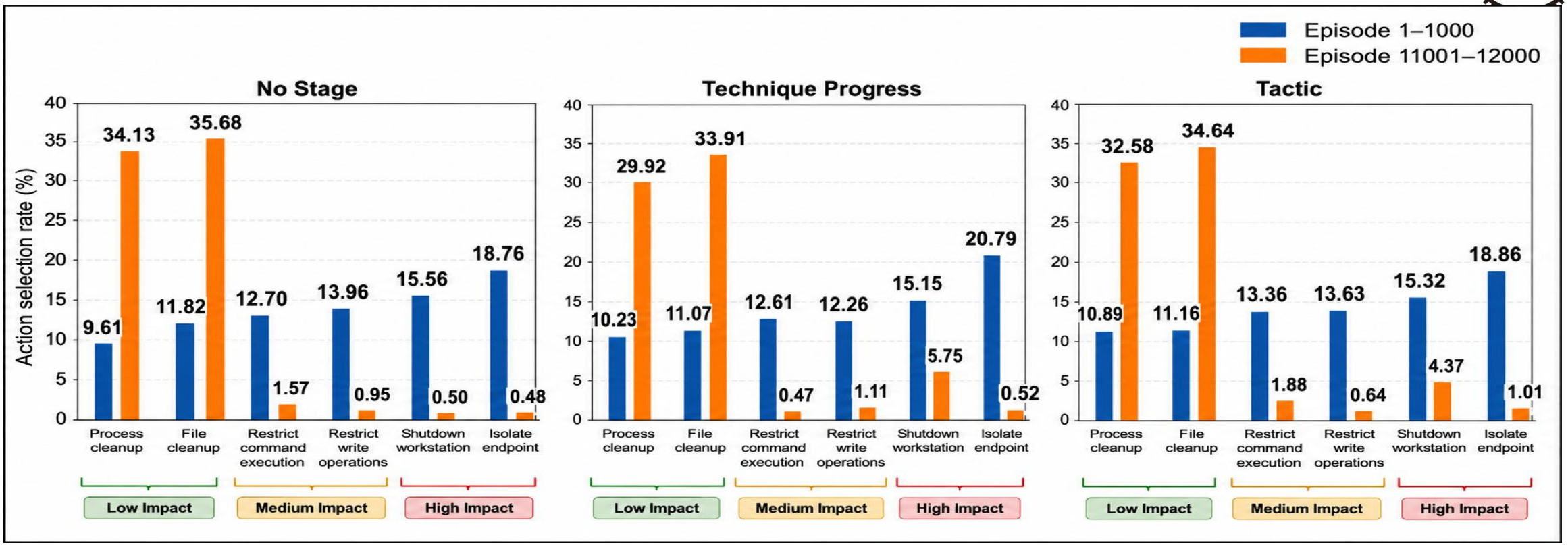


Figure. RL Action Distribution in Early and Late Training Across Configurations

- **Observation:**
  - **Process/File cleanup** become dominant in late training across all settings.
  - **No Stage** reduces high-impact actions the most: **Shutdown 15.56% → 0.50%**, **Isolate 18.76% → 0.48%**.
  - **Technique Progress** and **Tactic** keep more high-impact actions in late training, e.g. **Shutdown = 5.75%** and **4.37%**.
- **Insight:**
  - In early training, the agent still uses a relatively large share of **high-impact actions**.
  - In late training, **No Stage** becomes mostly **low-impact**, while **Technique Progress** and **Tactic** are more balanced.

Thank You

Questions & Discussion